

5.1 引言

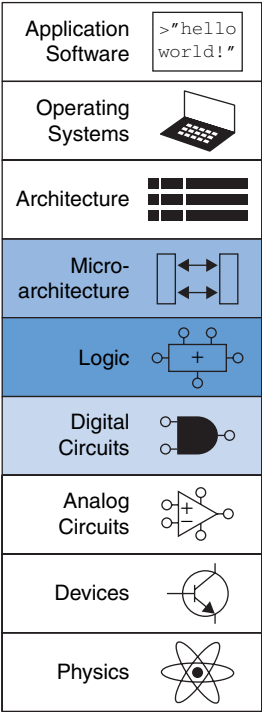
至此，我们已经通过布尔方程、原理图和硬件描述语言探讨了组合电路与时序电路的设计。本章将介绍数字系统中使用的更复杂的组合与时序构建模块，包括算术电路、计数器、移位寄存器、存储阵列和逻辑阵列。这些构建模块不仅本身具有实用价值，还体现了层次化、模块化和规律性的设计原则。它们通过层级化方式由更简单的组件（如逻辑门、多路选择器和译码器）组合而成。每个构建模块都具有明确的接口，在底层实现不重要时可视为黑箱。各构建模块的规整结构易于扩展至不同规模。在第七章中，我们将运用这些构建模块来构建微处理器。

5.2 算术电路

算术电路是计算机的核心构建模块。计算机和数字逻辑执行许多算术功能：加法、减法、比较、移位、乘法和除法。本节描述所有这些操作的硬件实现。

5.2.1 加法

加法是数字系统中最常见的操作之一。我们首先考虑如何将两个1位二进制数相加，然后扩展到N位二进制数。加法器也体现了速度与复杂度之间的权衡。



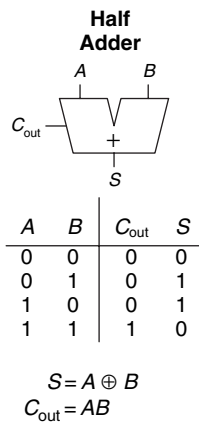


图5.1 1位半加器

0001
+0101

0110 1

图5.2 进位位

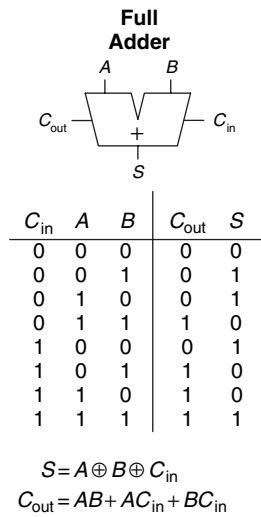


图5.3 1位全加器

半加器
我们首先构建一个 *half adder*。如图5.1所示，半加器有两个输入A和B，以及两个输出S和C_{out}。S是A与B的和。如果A和B均为1，则S为2，这无法用单个二进制位表示。相反，它通过下一列的进位输出C_{out}来指示。半加器可以由一个异或门和一个与门构建而成。

在多比特加法器中，C_{out}被加到或从 *carried in* 传递到下一个更高有效位。例如，在图5.2中，蓝色显示的进位位是第一列1比特加法的输出C_{out}，也是第二列加法的输入C_{in}。然而，半加器缺少一个C_{in}输入来接收前一列的C_{out}。下一节描述的 *full adder* 解决了这个问题。

全加器
如第2.1节所述，*full adder* 接受进位输入C_{in}，如图5.3所示。该图还展示了S和C_{out}的输出方程。

进位传播加法器
一个N位加法器将两个N位输入A和B以及进位输入C_{in}相加，产生一个N位结果S和进位输出C_{out}。它通常被称为 *carry propagate adder* (CPA)，因为一位的进位输出会传播到下一位。CPA的符号如图5.4所示；其绘制方式与全加器相同，只是A、B和S是总线而非单比特。三种常见的CPA实现分别称为行波进位加法器、超前进位加法器和前缀加法器。

纹波进位加法器
构建N位进位传播加法器的最简单方法是串联N个全加器。一级的C_{out}作为下一级的C_{in}，如图5.5所示的32位加法。这被称为 *ripple-carry adder*。这是模块化和规律性的良好应用：全加器模块被多次复用，以形成更大的系统。当N较大时，行波进位加法器具有速度慢的缺点。S₃₁依赖于C₃₀，而C₃₀依赖于C₂₉，C₂₉又依赖于C₂₈，依此类推，一直回溯到C_{in}，如图5.5中蓝色所示。我们说进位 *ripples* 通过进位链传播。加法器的延迟，

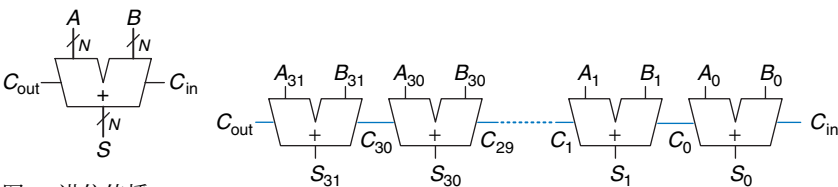


图5.4 进位传播加法器

图5.5 32位行波进位加法器

t_{ripple} 纹波直接随位数增长, 如公式5.1所示, 其中 t_{FA} 是一个全加器的延迟。

$$t_{\text{ripple}} = N t_{\text{FA}} \quad (5.1)$$

超前进位加法器

大型纹波进位加法器速度慢的根本原因在于进位信号必须逐位传播通过加法器的每一位。*carry- lookahead* 加法器 (CLA) 是另一种进位传播加法器, 它通过将加法器划分为 *blocks* 并设置电路, 在进位输入已知时快速确定块的进位输出来解决这一问题。因此, 它被称为在块之间进行 *look ahead*, 而不是等待进位在块内的所有全加器中逐位传播。例如, 一个32位加法器可以划分为八个4位块。

CLA使用 *generate* (G) 和 *propagate* (P) 信号来描述列或块如何确定进位输出。加法器的第 i 列如果产生与进位输入无关的进位输出, 则称其 *generate* 进位。如果 A_i 和 B_i 均为1, 则加法器的第 i 列保证生成进位 C_{i0} 。因此, 第 i 列的生成信号 G_i 计算为 $G_i = A_i \& B_i$ 。如果该列在存在进位输入时产生进位输出, 则称其 *propagate* 进位。第 i 列将在 A_i 或 B_i 为1时传播进位输入 C_{i-1} 。因此, $P_i = A_i \mid B_i$ 。利用这些定义, 我们可以重写加法器特定列的进位逻辑。加法器的第 i 列如果要么生成进位 G_i , 要么传播进位输入 $P_i C_{i-1}$, 则会产生进位输出 C_{i0} 。用方程表示为:

$$C_i = A_i B_i + (A_i + B_i) C_{i-1} = G_i + P_i C_{i-1} \quad (5.2)$$

生成和传播定义可扩展到多比特块。如果一个块独立于其进位输入而产生进位输出, 则称该块生成进位; 如果每当有进位输入到该块时, 它都会产生进位输出, 则称该块传播进位。我们定义跨越列 i 至 j 的1比特块生成信号 $G_{i:j}$ 和传播信号 $P_{i:j}$ 。

一个块产生进位, 如果最高有效列产生进位, 或者前一系列产生进位且最高有效列传播该进位, 依此类推。例如, 跨越第3列到第0列的块的生成逻辑为

$$G_{3:0} = G_3 + P_3(G_2 + P_2(G_1 + P_1 G_0)) \quad (5.3)$$

如果一个块中的所有列都传播进位, 则该块传播进位。例如, 跨越第3列到第0列的块的传播逻辑为

$$P_{3:0} = P_3 P_2 P_1 P_0 \quad (5.4)$$

原理图通常显示信号从左向右流动。算术电路打破了这一规则, 因为进位从右向左流动 (从最低有效列到最高有效列)。我们将最低有效列放在右侧, 最高有效列放在左侧, 这是因为我们习惯于这样查看和相加数字。



古往今来, 人们使用过许多工具进行算术。幼儿用手指计数 (有些成年人也会偷偷这样做)。中国人和巴比伦人早在公元前2400年就发明了算盘。1630年发明的计算尺一直使用到20世纪70年代, 当时科学手持计算器开始普及。如今, 计算机和数字计算器无处不在。下一个会是什么呢?

利用块生成和传播信号，我们可以通过块的进位输入 C_{i-1} 快速计算出块的进位输出 C_i 。

$$C_i = G_{i:j} + P_{i:j}C_{i-1} \tag{5.5}$$

图5.6(a)展示了一个由八个4位模块组成的32位超前进位加法器。每个模块包含一个4位行波进位加法器以及一些用于根据进位输入计算模块进位输出的超前进位逻辑，如图5.6(b)所示。为简洁起见，省略了使用信号 A_i 和 B_i 计算列生成和传播信号 G_i 和 P_i 所需的与门和或门。同样，超前进位加法器体现了模块化和规整性。

所有CLA模块同时计算1位列和模块的生成信号与传播信号。关键路径始于第一个CLA模块中计算 G_0 和 $G_{3:0}$ 。随后， C_{in} 通过每个模块中的与/或门直接传递至 C_{out} ，直至最后一个模块。具体而言，在所有列与模块的传播及生成信号计算完毕后， C_{in} 通过与/或门产生 C_3 ； C_3 随后通过其所在模块的与/或门产生 C_7 ； C_7 通过其所在模块的与/或门产生 C_{11} ，依此类推，直至

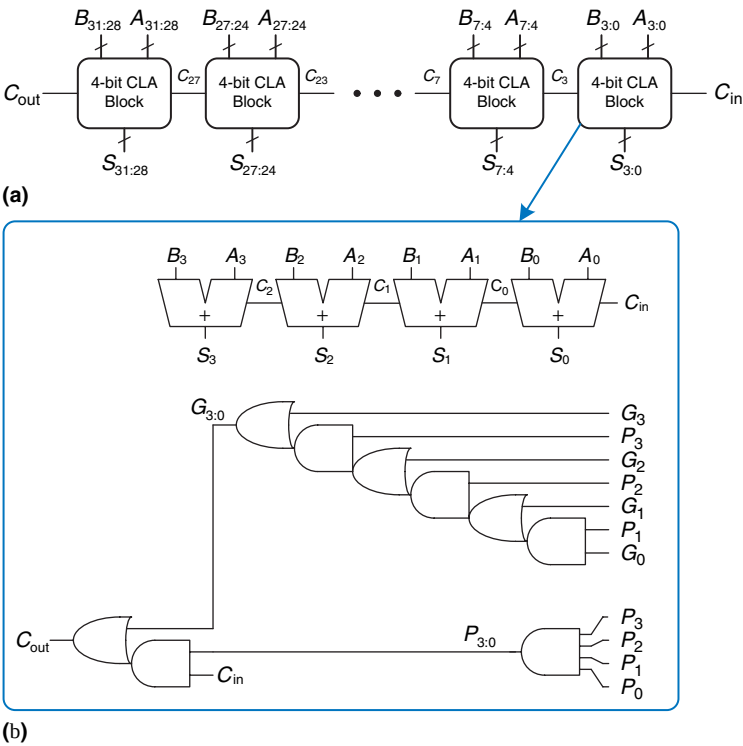


图5.6 (a) 32位超前进位加法器(CLA), (b) 4位CLA模块

C_{27} , 进位进入最后一个块。对于大型加法器而言, 这比等待进位逐位传播通过加法器的每个连续位要快得多。最后, 通过最后一个块的临界路径包含一个短的行波进位加法器。因此, 一个划分为 k 位块的 N 位加法器的延迟为

$$t_{CLA} = t_{pg} + t_{pg_block} + \left(\frac{N}{k} - 1 \right) t_{AND_OR} + k t_{FA} \quad (5.6)$$

其中 t_{pg} 是列传播与生成门 (单个AND或OR门) 产生 P_i 和 G_i 的延迟, t_{pg_block} 是查找 k 位块的块传播与生成信号 P_{ij} 和 G_{ij} 的延迟, t_{AND_OR} 是从 C_{in} 到 C_{out} 经过 k 位CLA块最终AND/OR逻辑的延迟。对于 $N > 16$, 超前进位加法器通常比行波进位加法器快得多。然而, 加法器延迟仍随 N 线性增加。

示例5.1 行波进位加法器与超前进位加法器 延迟

比较32位行波进位加法器和采用4位块的32位先行进位加法器的延迟。假设每个两输入门延迟为100 ps, 全加器延迟为300 ps。

根据公式5.1, 32位行波进位加法器的传播延迟为 $32 \times 300 \text{ ps} = 9.6 \text{ ns}$ 。

CLA具有 $t_{pg} = 100 \text{ ps}$, $t_{pg_block} = 6 \times 100 \text{ ps} = 600 \text{ ps}$, 以及 $t_{AND_OR} = 2 \times 100 \text{ ps} = 200 \text{ ps}$ 。根据公式5.6, 采用4位块的32位超前进位加法器的传播延迟因此为 $100 \text{ ps} + 600 \text{ ps} + (32/4 - 1) \times 200 \text{ ps} + (4 \times 300 \text{ ps}) = 3.3 \text{ ns}$, 比行波进位加法器快近三倍。

前缀加法器*

*Prefix adders*扩展超前进位加法器的生成和传播逻辑, 以更快地执行加法。它们首先计算每对列的 G 和 P , 然后计算4位块、8位块、16位块, 依此类推, 直到每一列的生成信号已知。和值则根据这些生成信号计算得出。

换句话说, 前缀加法器的策略是尽可能快地计算每一列 i 的进位 C_{i-1} , 然后利用该进位计算和。

$$S_i = (A_i \oplus B_i) \oplus C_{i-1} \quad (5.7)$$

定义列 $i = -1$ 以保存 C_{in} , 因此 $G_{-1} = C_{in}$ 和 $P_{-1} = 0$ 。然后, $C_{i-1} = G_{i-1:-1}$ 因为如果跨越列 $i-1$ 至 -1 的块产生进位, 则列 $i-1$ 将产生进位。所产生的进位

要么在列*i*-1中生成，要么在之前的列中生成并传播。因此，我们将公式5.7重写为

$$S_i = (A_i \oplus B_i) \oplus G_{i-1:-1} \tag{5.8}$$

早期计算机使用行波进位加法器，因为当时元件昂贵，而行波进位加法器所需硬件最少。如今几乎所有现代个人计算机都在关键路径上采用前缀加法器，因为晶体管成本低廉，且速度至关重要。

因此，主要挑战在于快速计算所有块生成信号 $G_{-1:-1}$ 、 $G_{0:-1}$ 、 $G_{1:-1}$ 、 $G_{2:-1}$ 、 $\cdots$ 、 $G_{N-2:-1}$ 。这些信号与 $P_{-1:-1}$ 、 $P_{0:-1}$ 、 $P_{1:-1}$ 、 $P_{2:-1}$ 、 $\cdots$ 、 $P_{N-2:-1}$ 一起被称为*prefixes*。

图5.7展示了一个*N*=16位前缀加法器。该加法器首先使用*precomputation*通过AND和OR门从 A_i 和 B_i 为每一列生成 P_i 和 G_i 。然后利用 $\log_2 N$ =4级黑色单元形成 $G_{i:j}$ 和 $P_{i:j}$ 的前缀。黑色单元从覆盖位*i*:*k*的块的上半部分和覆盖位*k*-1:*j*的下半部分接收输入，并通过以下方程将这些部分组合，生成覆盖位*i*:*j*的整个块的进位生成信号和进位传播信号。

$$G_{i:j} = G_{i:k} + P_{i:k}G_{k-1:j} \tag{5.9}$$

$$P_{i:j} = P_{i:k} P_{k-1:j} \tag{5.10}$$

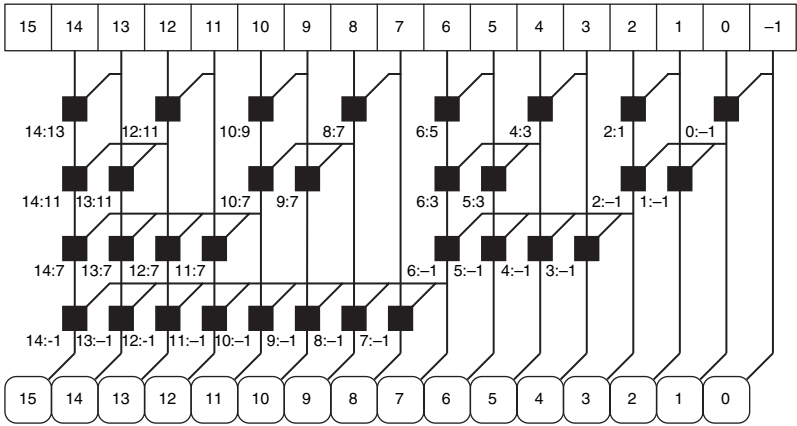
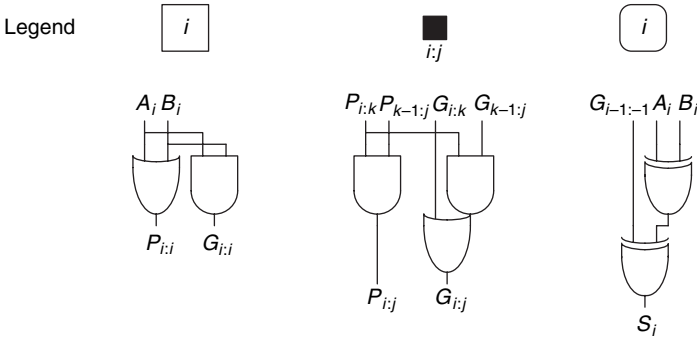


图5.7 16位前缀加法器



换言之，一个跨越比特 $i;j$ 的块将在以下情况下产生进位：如果高位部分($i:k$)产生进位，或者高位部分传播了低位部分($k-1;j$)产生的进位。该块将传播进位当且仅当高位和低位部分都传播进位。最后，前缀加法器使用公式5.8计算和。

总之，前缀加法器实现的延迟随加法器列数呈对数增长而非线性增长。这种加速效果显著，尤其对于32位或更多位的加法器，但其代价是硬件复杂度高于简单的超前进位加法器。黑色单元网络被称为*prefix tree*。

使用前缀树在时间随输入数量对数增长的情况下执行计算的一般原理是一种强大的技术。通过一些巧思，它可以应用于许多其他类型的电路（例如，参见练习5.7）。

对于 N 位前缀加法器，关键路径包括预计算 P_i 和 G_i ，随后经过 $\log_2 N$ 级黑色前缀单元以获得所有前缀。然后 $G_{i-1:-1}$ 通过底部的最终XOR门计算 S_{i0} 。从数学上讲， N 位前缀加法器的延迟为

$$t_{PA} = t_{pg} + \log_2 N(t_{pg_prefix}) + t_{XOR} \quad (5.11)$$

其中 t_{pg_prefix} 是黑色前缀单元的延迟。

注意前缀加法器中最后一个（求和）框的延迟是一个异或门延迟（而非两个），因为 $A_i \oplus B_i$ 的计算已在更早完成，这是由于 A_i 和 B_i 当时已经可用。

示例5.2 前缀加法器延迟

计算一个32位前缀加法器的延迟。假设每个两输入门延迟为100 ps。

解 每个黑色前缀单元 t_{pg_prefix} 的传播延迟为200 ps（即两个门延迟）。因此，使用公式5.11，32位前缀加法器的传播延迟为 $100 \text{ ps} + \log_2(32) \times 200 \text{ ps} + 100 \text{ ps} = 1.2 \text{ ns}$ ，这比示例5.1中的超前进位加法器快约三倍，比行波进位加法器快约八倍。在实际应用中，优势虽没有如此显著，但前缀加法器仍然比替代方案快得多。

综合运用

本节介绍了半加器、全加器以及三种进位传播加法器：行波进位加法器、超前进位加法器和前缀加法器。更快的加法器需要更多硬件，因此成本更高且功耗更大。在为设计选择合适的加法器时，必须权衡这些因素。

硬件描述语言提供+操作来指定CPA。现代综合工具从多种可能的实现中进行选择，

HDL示例5.1 进位传播加法器（CPA）

SystemVerilog	VHDL
<pre>module adder #(parameter N = 8) (input logic [N-1:0] a, b, input logic cin, output logic [N-1:0] s, output logic cout); assign {cout, s} = a + b + cin; endmodule</pre>	<pre>library IEEE; use IEEE.STD_LOGIC_1164.ALL; use IEEE.NUMERIC_STD.UNSIGNED.ALL; entity adder is generic(N: integer := 8); port(a, b: in STD_LOGIC_VECTOR(N-1 downto 0); cin: in STD_LOGIC; s: out STD_LOGIC_VECTOR(N-1 downto 0); cout: out STD_LOGIC); end; architecture synth of adder is signal result: STD_LOGIC_VECTOR(N downto 0); begin result <= ("0" & a) + ("0" & b) + cin; s <= result(N-1 downto 0); cout <= result(N); end;</pre>

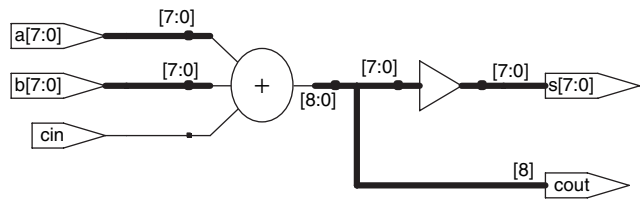


图5.8 综合加法器

选择满足速度要求的最便宜（最小）设计。这大大简化了设计者的工作。HDL示例5.1描述了带有进位输入和输出的CPA，图5.8显示了生成的硬件。

5.2.2 减法

回顾第1.4.6节，加法器可以使用二进制补码表示法对正数和负数进行加法运算。减法几乎同样简单：翻转（即取反）第二个数的符号，然后相加。翻转二进制补码数的符号是通过逐位取反再加1来实现的。

要计算 $Y = A - B$ ，首先创建 B 的二进制补码：将 B 的各位取反得到 $\bar{B}$ ，再加1得到 $-\bar{B} = \bar{B} + 1$ 。将此值加到 A 上得到 $Y = A + \bar{B} + 1 = A - B$ 。该求和可通过单个CPA实现，即把 $A + \bar{B}$ 与 $C_{in} = 1$ 相加。图5.9展示了减法器的符号以及执行 $Y = A - B$ 的底层硬件。HDL示例5.2描述了减法器，图5.10展示了生成的硬件。

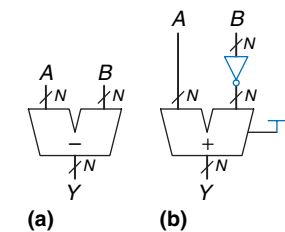


图5.9 减法器：(a) 符号，(b) 实现

HDL示例5.2 减法器

SystemVerilog

```
module subtractor #(parameter N = 8)
    (input  logic [N-1:0] a, b,
     output logic [N-1:0] y);

    assign y = a - b;
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD_UNSIGNED.ALL;

entity subtractor is
    generic(N: integer := 8);
    port(a, b: in  STD_LOGIC_VECTOR(N-1 downto 0);
         y:      out STD_LOGIC_VECTOR(N-1 downto 0));
end;

architecture synth of subtractor is
begin
    y <= a - b;
end;
```

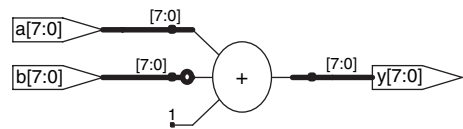


Figure 5.10 Synthesized subtractor

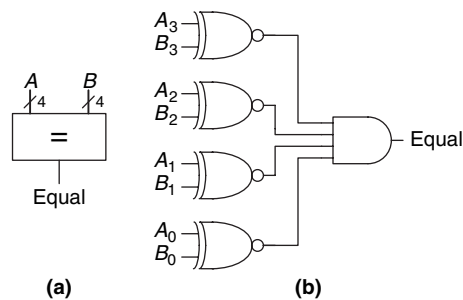


Figure 5.11 4-bit equality comparator: (a) symbol, (b) implementation

5.2.3 比较器

一个comparator判断两个二进制数是否相等，或者一个是否大于或小于另一个。比较器接收两个N位的二进制数A和B，并输出1位的比较结果。

一个equality comparator产生单一输出，指示A是否等于B（ $A == B$ ）。一个magnitude comparator产生一个或多个输出，指示A和B的相对值。

相等比较器是一种较为简单的硬件。图5.11展示了4位相等比较器的符号和实现方式。它首先使用XNOR门检查A和B每一列中对应位是否相等。如果所有列都相等，则这两个数相等。

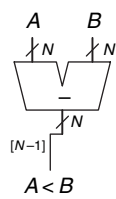


图5.12 N位有符号比较器

有符号数的大小比较通常通过计算 $A - B$ 并查看结果的符号位（最高有效位）来完成，如图5.12所示。如果结果为负（即符号位为1），则 A 小于 B 。否则， A 大于或等于 B 。然而，这种比较器在发生溢出时会出错。习题5.9和5.10探讨了这一局限性及其解决方法。HDL示例5.3展示了如何对无符号数使用各种比较操作，图5.13显示了生成的硬件。

HDL示例5.3 比较器

SystemVerilog

```
module comparators #(parameter N = 8)
    (input  logic [N-1:0] a, b,
     output logic eq, neq, lt, lte, gt, gte);

    assign eq  = (a == b);
    assign neq = (a != b);
    assign lt  = (a < b);
    assign lte = (a <= b);
    assign gt  = (a > b);
    assign gte = (a >= b);
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;

entity comparators is
    generic(N: integer := 8);
    port(a, b: in STD_LOGIC_VECTOR(N-1 downto 0);
         eq, neq, lt, lte, gt, gte: out STD_LOGIC);
end;

architecture synth of comparators is
begin
    eq <= '1' when (a = b) else '0';
    neq <= '1' when (a /= b) else '0';
    lt <= '1' when (a < b) else '0';
    lte <= '1' when (a <= b) else '0';
    gt <= '1' when (a > b) else '0';
    gte <= '1' when (a >= b) else '0';
end;
```

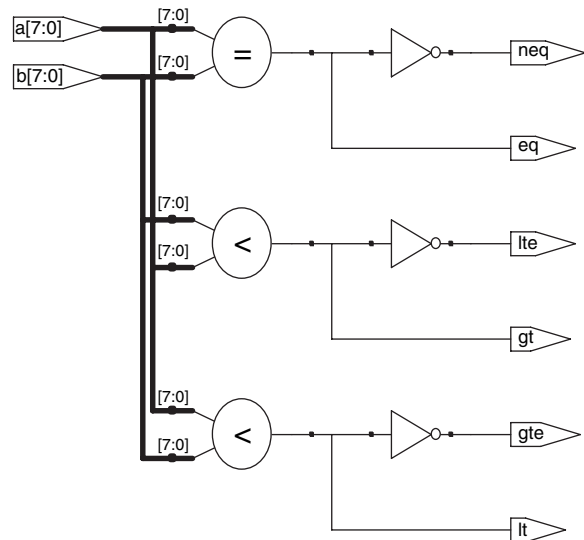


图5.13 合成比较器

表5.1 ALU操作

<i>ALUControl</i> _{1:0}	Function
00	Add
01	Subtract
10	AND
11	OR

5.2.4 算术逻辑单元

Arithmetic/Logical Unit (ALU) 将多种数学和逻辑运算整合到单个单元中。例如，典型的ALU可执行加法、减法、与运算和或运算。ALU构成了大多数计算机系统的核心。

图5.14展示了一个N位ALU的符号，其输入和输出为N位。该ALU接收一个2位控制信号*ALUControl*，用于指定执行何种功能。控制信号通常以蓝色显示，以区别于数据。表5.1列出了ALU可以执行的典型功能。

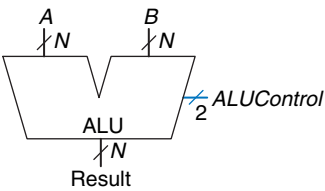


图5.14 ALU符号

图5.15展示了ALU的一种实现方式。该ALU包含一个N位加法器、N个二输入与门和或门，此外还包含反相器。

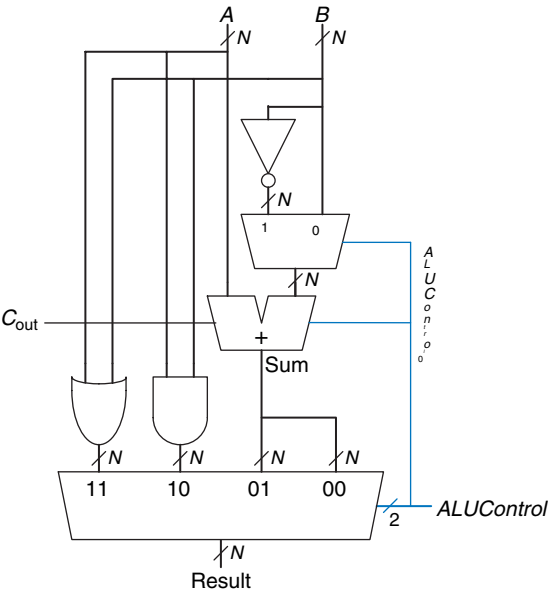


图5.15 N位ALU

以及一个多路选择器，当 $ALUControl_0$ 有效时对输入 B 进行取反。一个4:1多路选择器根据 $ALUControl$ 选择所需的功能。

更具体地说，如果 $ALUControl = 00$ ，输出多路选择器选择 $A + B$ 。如果 $ALUControl = 01$ ， ALU 计算 $A - B$ 。（回顾第5.2.2节，在二进制补码算术中， $B + 1 = -B$ 。由于 $ALUControl_0$ 为1，加法器接收输入 A 和 B 以及一个有效的进位输入，从而执行减法： $A + B + 1 = A - B$ 。）如果 $ALUControl = 10$ ， ALU 计算 A 与 B 。如果 $ALUControl = 11$ ， ALU 执行 A 或 B 。

某些 ALU 会生成额外的输出，称为 $flags$ ，用于指示 ALU 输出的相关信息。图5.16展示了带有4位 $Flags$ 输出的 ALU 符号。如图5.17中该 ALU 的原理图所示， $Flags$ 输出由 N 、 Z 、 C 和 V 标志组成，分别表示 ALU 输出 $Result$ 为负或零，或者加法器产生了进位或溢出。回顾一下，二进制补码数的最高有效位若为1则表示负数，否则为0。因此， N （ N 负）标志连接到 ALU 输出的最高有效位 $Result_{31}$ 。当 $Result$ 的所有位均为0时， Z （ Z 零）标志被置位，这是由图5.17中的 N 位或非门检测的。当加法器产生进位输出 and 且 ALU 正在执行加法或减法（由 $ALUControl_1 = 0$ 指示）时， C （ C 进位输出）标志被置位。

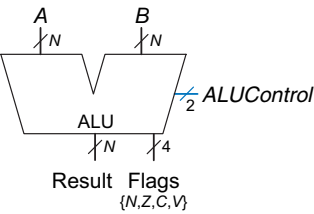


图5.16 带输出标志的ALU符号

溢出检测，如图5.17左侧所示，更为棘手。回顾第1.4.6节，当加法运算

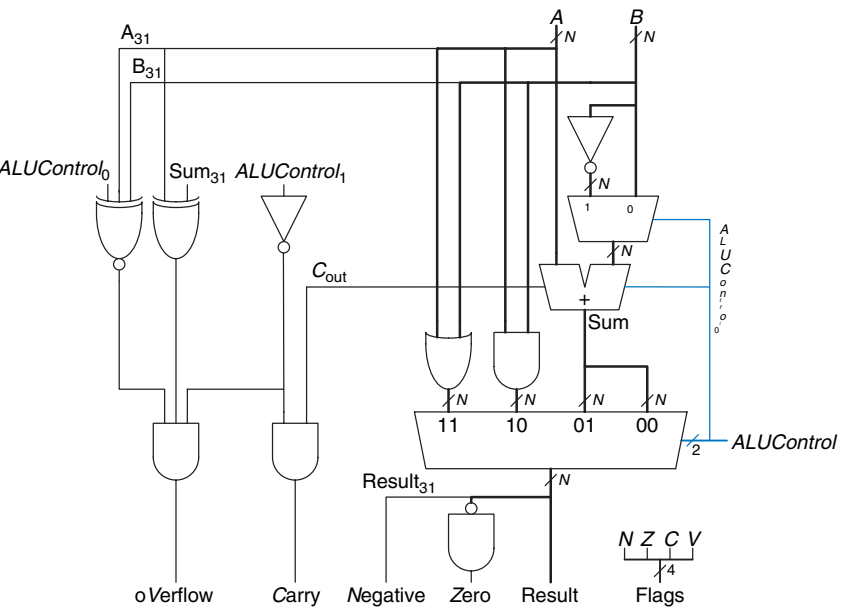


图5.17 带输出标志的N位ALU

表5.2 有符号与无符号比较

Comparison	Signed	Unsigned
=	Z	Z
$\neq$	$\bar{Z}$	$\bar{Z}$
<	$N \oplus V$	$\bar{C}$
$\leq$	$Z + (N \oplus V)$	$Z + \bar{C}$
>	$\bar{Z} \bullet (\bar{N} \oplus \bar{V})$	$\bar{Z} \bullet C$
$\geq$	$(\bar{N} \oplus \bar{V})$	C

两个相同符号的数会产生一个相反符号的结果。因此，当以下三个条件同时成立时， V （溢出）被断言：（1）ALU正在执行加法或减法（ $ALUControl_1 = 0$ ），（2） A 和 Sum 具有相反的符号（由异或门检测），以及（3）溢出是可能的。也就是说，由同或门检测，要么 A 和 B 具有相同的符号且加法器正在执行加法（ $ALUControl_0 = 0$ ），要么 A 和 B 具有相反的符号且加法器正在执行减法（ $ALUControl_0 = 1$ ）。三输入与门检测到所有三个条件同时成立时，断言 V 。

如表 5.2所示，ALU标志也可用于比较。要比较输入 A 和 B ，ALU会计算 $A - B$ 并查看标志位。若 Z 有效，则结果为0，因此 $A = B$ 。否则， A 不等于 B 。

*Magnitude comparison*较为复杂，取决于数字是有符号还是无符号。例如，要确定 $A < B$ ，我们计算 $A - B$ 并检查结果是否为负。如果数字是无符号的，则当没有进位时结果为负。¹如果数字是有符号的，我们不能依赖进位，因为小的负数与大的正无符号数表示方式相同。相反，我们直接计算 $A - B$ 并查看答案是否为负，这由 N 标志指示。然而，如果发生溢出， N 标志将不正确。因此， A 小于 B 的条件是 $N \oplus V$ （换句话说，如果答案为负且没有溢出，或者答案为正但发生了溢出）。总结一下，我们定义 L （ L 小于信号）在 $A < B$ 时为真。对于无符号数， $L = C$ 。对于有符号

¹You can check this by trying some numbers. Alternatively, note that reversing the sign (i.e., taking the two's complement) of N -bit numbers for subtraction produces $-B = \bar{B} + 1 = 2^N - B$. Now, $A + (-B) = 2^N + A - B$. This will have a carry out (a 1 in column N) if $A \geq B$ and, hence, no carry out if $A < B$.

数字, $L = N \oplus V$ 。其余检查较为简单。小于或等于 ($\leq$) 为 L 或 Z , 因为 L 表示小于, Z 表示等于。大于或等于 ($\geq$) 是小于的反义: L 。大于 ($>$) 由大于或等于但不等于表示: L 且 Z 。

示例5.3 比较

考虑4位数字 $A = 1111$ 和 $B = 0010$ 。判断 $A < B$, 首先将数字解释为无符号数 (15 和 2), 然后解释为有符号数 (-1 和 2)。

解 计算 $A - B = A + B + 1 = 11\bar{1}1 + 1101 + 1 = 11101$ 。进位输出 C 为 1, 如蓝色所示。 N 标志位为 1, 如斜体所示。 V 标志位为 0, 因为结果与 A 的符号位相同。 Z 标志位为 0, 因为答案不是 0000。

对于无符号比较, $L = \sim C = 0$, 因为 15 不小于 2。对于有符号比较, $L = N \oplus V = 1$, 因为 -1 小于 2。

某些ALU还实现了一条名为 *set if less than* (SLT) 的指令。当 $A < B$ 时, 结果为 $Result = 1$ 。否则, 结果为 $Result = 0$ 。这对于无法访问ALU标志的计算机来说非常方便, 因为它本质上将标志信息存储在结果中。SLT通常将输入视为有符号数。另一种变体 (SLTU) 则将输入视为无符号数。这种基本ALU存在许多变体, 支持其他功能, 例如NOT、XOR或XNOR。一个 N 位ALU的HDL (包括支持SLT和输出标志的版本) 留待练习5.11至5.14完成。

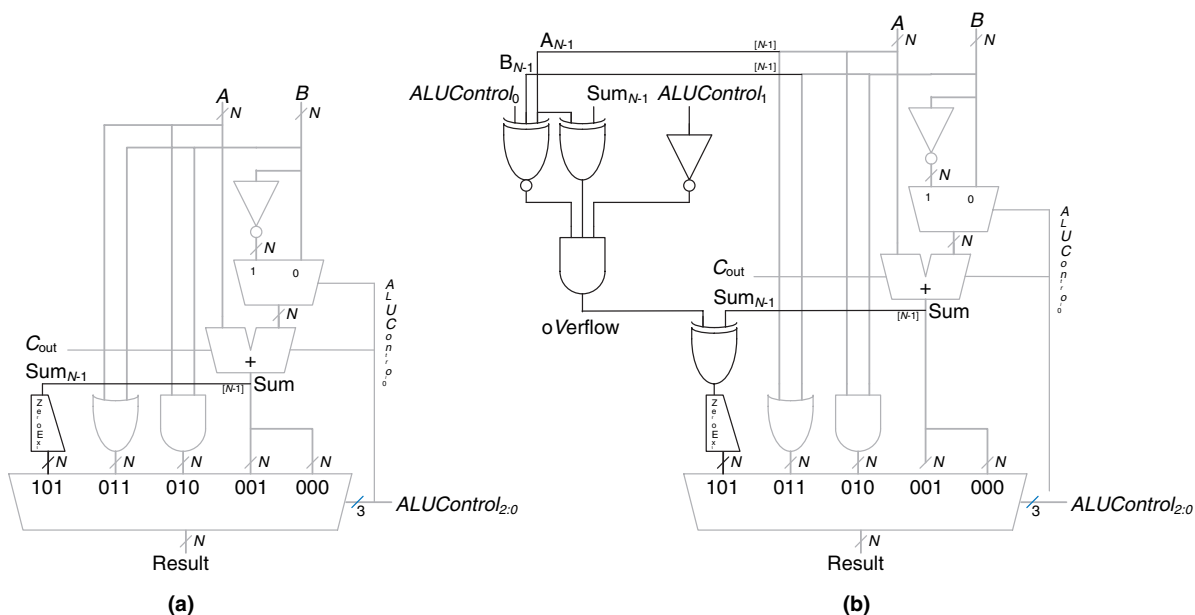
示例5.4 扩展ALU以处理SLT

扩展ALU以处理小于则置位 (SLT) 操作。

表5.3 用于SLT的ALU扩展操作

$ALUControl_{2:0}$	Function
000	Add
001	Subtract
010	AND
011	OR
101	SLT

解决方案 要为ALU添加另一个功能, 我们必须扩展多路复用器使其具有五个输入。我们通过执行 $A - B$ 来判断 A 是否小于 B ; 如果结果为负, 则 A 小于 B 。表5.3显示了处理SLT的更新后的 $ALUControl$ 信号, 图5.18(a)显示了扩展后的电路, 其中控制信号的变化以蓝色高亮显示, 其他变化以黑色显示。我们使用 $ALUControl = 101$ 进行SLT操作, 并利用将 $ALUControl_0 =$ 设为1会使加法器执行 $A - B$ 这一事实。当 Sum_{N-1} 为1时, $A - B$ 的结果为负, 且 A 小于 B 。因此, 我们对 Sum_{N-1} 进行零扩展, 并将其输入到101多路复用器的输入端, 以完成SLT操作。但请注意, 此实现未考虑溢出。当发生溢出时, Sum 的符号将不正确。因此, 我们将 Sum 的符号位与溢出信号 V 进行异或运算, 以正确指示负的 Sum , 如图5.18(b)所示。



5.2.5 移位器和旋转器

Shiftersrotators 移动位，并通过2的幂进行乘法或除法。顾名思义，移位器将二进制数向左或向右移动指定的位数。存在几种常用的移位器：

逻辑移位器——将数字向左或向右移动，空位补0。示例：11001 >> 2 = 00110；11001 << 2 = 00100。算术移位器——与逻辑移位器相同，但在右移时，最高有效位用原最高有效位（msb）的副本填充。这对于有符号数的乘除运算很有用（参见5.2.6节和5.2.7节）。算术左移与逻辑左移相同。

示例: $11001 \ggg 2 = 11110$; $11001 \lll 2 = 00100$ 。运算符 $\lll$ 、 $\ggg$ 和 $\ggg$ 通常分别表示左移、逻辑右移和算术右移。

旋转器——循环旋转一个数字，使得空位被从另一端移出的位填充。

示例：11001 ROR 2 = 01110；11001 ROL 2 = 00111。ROR = 循环右移；ROL = 循环左移。

一个N位移位器可以由N N:1个多路复用器构建。输入根据 $\log_2 N$ 位选择线的值，移位0到N-1位。图5.19展示了4位移位器的符号和硬件。根据2位移位量 $shamt_{1:0}$ 的值，输出Y接收输入A移位0到3位的结果。对于所有移位器，当 $shamt_{1:0}$ 为00时， $Y = A$ 。练习5.22涵盖旋转器的设计。

左移是乘法的一种特殊情况。左移N位相当于将数字乘以 2^N 。例如， $000011_2 \ll 4 = 110000_2$ 等同于 $3_{10} \times 2^4 = 48_{10}$ 。

算术右移是除法的一种特殊情况。将数字右移N位相当于除以 2^N 。例如， $111100_2 \gg 2 = 1111_2$ 等价于 $-4_{10}/2^2 = -1_{10}$ 。

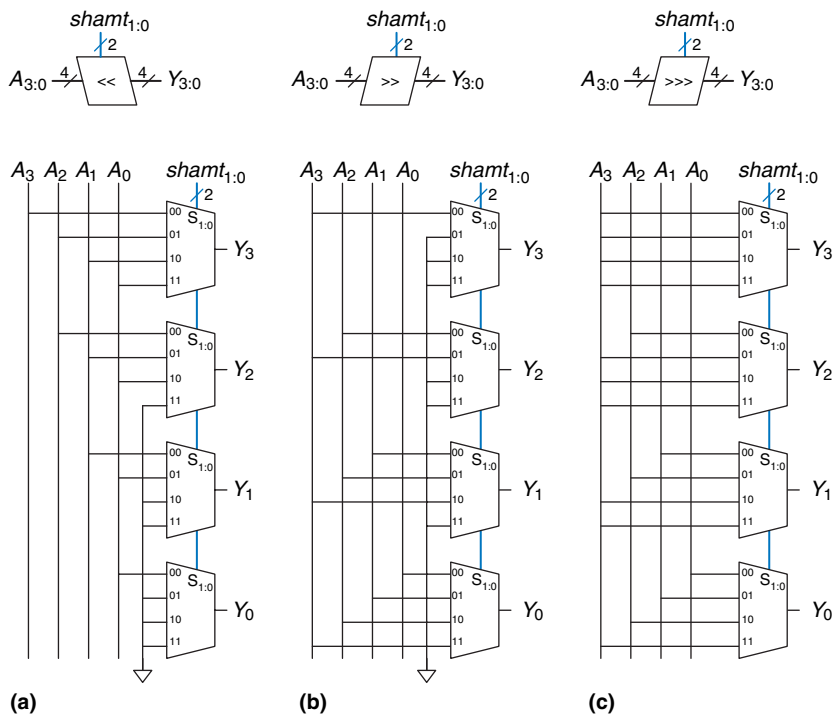


图5.19 4位移位器：(a) 左移，(b) 逻辑右移，(c) 算术右移

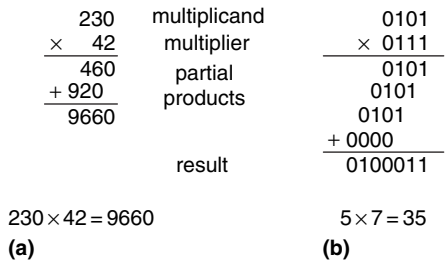


图5.20 乘法：(a)十进制，
(b)二进制

5.2.6 乘法*

无符号二进制数的乘法与十进制乘法类似，但仅涉及1和0。图5.20比较了十进制和二进制中的乘法。在这两种情况下，*partial products*是通过将乘数的一位与整个被乘数相乘而形成的。移位后的部分乘积相加得到最终结果。

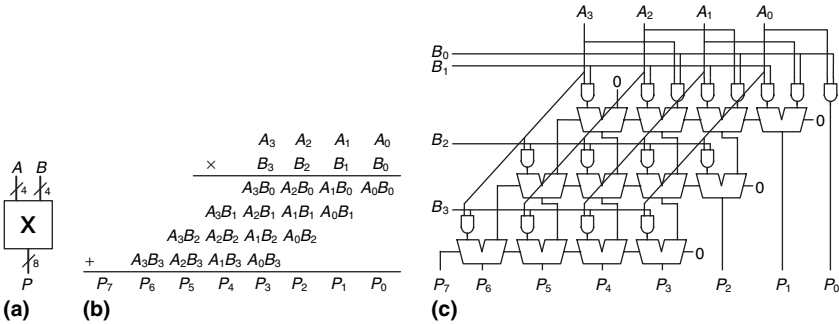
一般而言，一个 $N \times N$ 乘法器将两个 N 位数字相乘，并产生一个 $2N$ 位的结果。二进制乘法中的部分积要么是被乘数，要么是全0。1位二进制数的乘法等同于与运算，因此使用与门来形成部分积。

有符号乘法与无符号乘法不同。例如，考虑 $0xFE \times 0xFD$ 。如果将这些8位数字解释为有符号整数，它们表示-2和-3，因此16位乘积为 $0x0006$ 。如果将这些数字解释为无符号整数，16位乘积为 $0xFB06$ 。注意，无论哪种情况，最低有效字节都是 $0x06$ 。

图5.21展示了一个无符号 4×4 乘法器的符号、功能及实现。该无符号乘法器接收被乘数和乘数 A 与 B ，并生成乘积 P 。图5.21(b)展示了部分积的形成方式。每个部分积由单个乘数位 (B_3 、 B_2 、 B_1 或 B_0) 与被乘数位 (A_3 、 A_2 、 A_1 、 A_0) 进行AND运算得到。对于 N 位操作数，共有 N 个部分积和 $N - 1$ 级1位加法器。例如，在 4×4 乘法器中，第一行的部分积为 B_0 与 (A_3 、 A_2 、 A_1 、 A_0) 的AND结果。该部分积与移位后的第二行部分积 (B_1 与 (A_3 、 A_2 、 A_1 、 A_0) 的AND结果) 相加。后续的AND门与加法器行继续生成并累加剩余的部分积。

有符号和无符号乘法器的HDL代码见HDL示例4.33。与加法器类似，存在多种不同速度/成本权衡的乘法器设计。综合工具会根据时序约束选择最合适的设计。

图5.21 4 × 4乘法器：(a) 符号，(b) 功能，(c) 实现



*multiply accumulate*操作将两个数相乘，然后与第三个数（通常是累加值）相加。这些操作也称为MAC，常用于*digital signal processing*（数字信号处理）算法中，例如需要乘积求和的傅里叶变换。

5.2.7 除法*

对于范围在 $[0, 2^N - 1]$ 内的N位无符号数，可以使用以下算法执行二进制除法：

$$\begin{aligned} R' &= 0 \text{ 对于 } i = N-1 \text{ 到 } 0 \quad R = \{R' \ll 1, A_i\} \quad D = R - B \\ \text{如果 } D < 0 \text{ 则 } Q_i &= 0, R' = R \quad // R < B \quad \text{否则} \quad Q_i \\ &= 1, R' = D \quad // R \geq B \quad R = R' \end{aligned}$$

partial remainder R 被初始化为 0 ($R' = 0$)，被除数 A 的最高有效位成为 R 的最低有效位 ($R = \{R' \ll 1, A_i\}$)。从该部分余数中减去除数 B ，以判断是否可整除 ($D = R - B$)。如果差值 D 为负（即 D 的符号位为 1），则商位 Q_i 为 0，并丢弃该差值。否则， Q_i 为 1，并将部分余数更新为差值。无论哪种情况，部分余数随后加倍（左移一位）， A 的下一个最高有效位成为 R 的最低有效位，并重复该过程。结果满足 $A/B = Q + R/B$ 。

图5.22展示了一个4位阵列除法器的原理图。该除法器计算 A/B ，并产生商 Q 和余数 R 。图例显示了阵列中每个模块的符号和原理图。

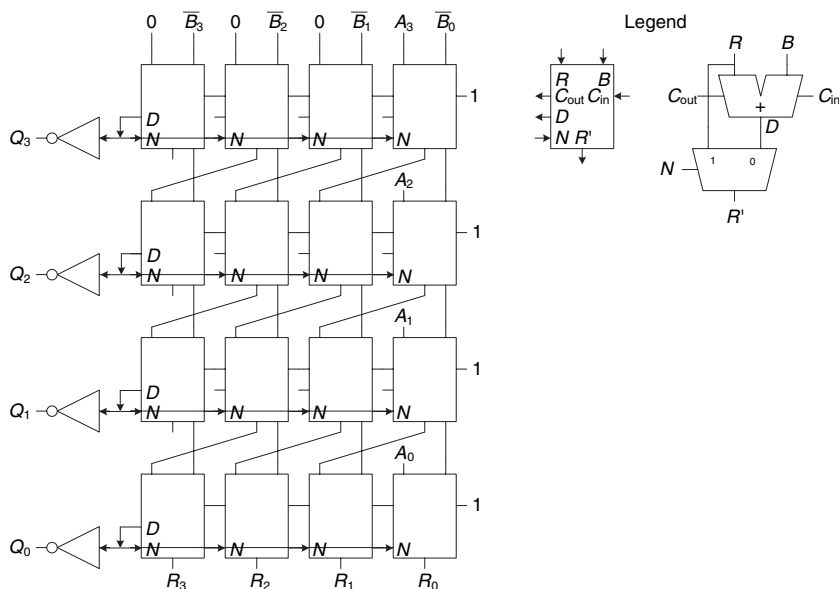


Figure 5.22 Array divider

除法器。每一行执行除法算法的一次迭代。具体来说，每一行计算差值 $D = R - B$ 。（回想一下， $R + B + 1 = R - B$ ）。多路复用器选择信号 N （用于 N 负）在某一行的差值 D 为负时接收1。因此 N 由 D 的最高有效位驱动，当差值为负时该位为1。每个商位 (Q_i) 在 D 为负时为0，否则为1。如果差值为负，多路复用器将 R 传递到下一行，否则传递 D 。下一行将新的部分余数左移一位，附加 A 的下一个最高有效位，然后重复该过程。

N 位阵列除法器的延迟与 N^2 成正比增加，因为进位必须在确定符号且多路选择器选择 R 或 D 之前，逐行通过所有 N 级。这一过程对所有 N 行重复。除法在硬件中是一种缓慢且昂贵的操作；因此，应尽可能少用。

5.2.8 扩展阅读

计算机算术可以成为一整本书的主题。*Digital Arithmetic* (Ercegovic 和 Lang 著) 是对整个领域的极佳概述。*CMOS VLSI Design* (Weste 和 Harris 著) 则涵盖了算术运算的高性能电路设计。

5.3 数制

计算机既处理整数也处理分数。到目前为止，我们仅考虑了如第1.4节所介绍的表示有符号或无符号整数。本节将介绍能够表示有理数的定点数和浮点数系统。定点数类似于十进制数，其中一些位表示整数部分，其余位表示分数部分。浮点数则类似于科学记数法，包含尾数和指数。

5.3.1 定点数系统

*Fixed-point notation*在整数位和小数位之间隐含了一个*binary point*，类似于普通十进制数中整数位和小数位之间的小数点。例如，图5.23(a)展示了一个具有四个整数位和四个小数位的定点数。图5.23(b)用蓝色显示了隐含的二进制小数点，图5.23(c)展示了等效的十进制数值。

(a) 01101100 (b) 0110.1100 (
c) $2^2 + 2^1 + 2^{-1} + 2^{-2} = 6.75$

图5.23 6.75的定点表示法，包含四个整数位和四个小数位

有符号定点数可以使用二进制补码或符号/幅度表示法。图5.24展示了使用这两种表示法、以四个整数位和四个小数位表示-2.375的定点数形式。为清晰起见，隐式二进制小数点以蓝色显示。在符号/幅度形式中，最高有效位用于指示符号。二进制补码表示是通过对绝对值各位取反，然后在最低有效（最右侧）位上加1形成的。在此例中，最低有效位位于 2^{-4} 列。

(a) 0010.0110 (
b) 1010.0110 (c
) 1101.1010

图5.24 -2.375的定点表示：(a) 绝对值，(b) 符号与数值，(c) 二进制补码

与所有二进制数表示法一样，定点数也只是一组比特位。除了通过解释该数字的人员之间的约定，无法得知二进制小数点的存在。

通常，我们使用 $Ua.b$ 表示一个无符号定点数，其中包含 a 个整数位和 b 个小数位。 $Qa.b$ 表示一个有符号（二进制补码）定点数，其中包含 a 个整数位（包括符号位）和 b 个小数位。

示例5.5 定点数的算术运算

使用Q4.4定点数计算 $0.75 + (-0.625)$ 。

解法 首先，将第二个数的数值 0.625 转换为定点二进制表示法。 $0.625 \geq 2^{-1}$ ，因此在 2^{-1} 列中有一个 1，剩余 $0.625 - 0.5 = 0.125$ 。由于 $0.125 < 2^{-2}$ ，在 2^{-2} 列中有一个 0。由于

0000.1010

1111.0101

+

1

1111.0110

Binary Magnitude

One's Complement

Add 1

Two's Complement

图5.25 定点二进制补码转换

0000.1100

+

1111.0110

10000.0010

(a)

0.75

+

(-0.625)

0.125

(b)

图5.26 加法：(a)二进制定点，(b)十进制等价

0.125 ≥ 2⁻³，在2⁻³列中有一个1，剩下0.125 − 0.125 = 0。因此，2⁻⁴列中必须有一个0。综合起来，0.625₁₀ = 0000.1010₂。

对有符号数使用二进制补码表示，以确保加法运算正确。图5.25展示了将−0.625转换为定点二进制补码表示的过程。

图5.26展示了定点二进制加法及其十进制等效值以供比较。注意，在图5.26(a)的二进制定点加法中，前导的1被从8位结果中丢弃。

5.3.2 浮点数系统*

浮点数类似于科学记数法。它们规避了整数和小数位固定的限制，从而能够表示非常大和非常小的数字。与科学记数法一样，浮点数包含*sign*、*mantissa* (M)、*base* (B) 和*exponent* (E)，如图5.27所示。例如，数字4.1 × 10³ 是4100的十进制科学记数法，其尾数为4.1，基数为10，指数为3。小数点*floats*位于最高有效位之后。浮点数是基数为2的二进制尾数。32位用于表示1个符号位、8个指数位和23个尾数位。

定点数系统广泛应用于数字信号处理（DSP）、图形和机器学习应用中，因为其计算速度比浮点系统更快且功耗更低。Q1.15（也称为Q15）是最常见的格式，用于存储范围在(−1, 1)内的有符号数，精度为15位。Q1.31（简称Q31）有时用于更高精度的中间结果，例如快速傅里叶变换中。U8.8有时用于模数转换器（ADC）采样的传感器读数。请注意，所有这些格式都打包为16位或32位字，以便在计算机内存中高效存储，而计算机内存宽度通常为2的幂。

示例5.6 32位浮点数

显示十进制数228的浮点数表示。

解法 首先，将十进制数转换为二进制：228₁₀ = 11100100₂ = 1.11001₂ × 2⁷。图5.28展示了32位编码，后续将对其进行效率优化。符号位为正（0），8位指数位给出数值7，剩余23位为尾数。

±M × B^E

图5.27 浮点数

图5.28 32位浮点 版本1

1 bit	8 bits	23 bits
0	00000111	111 0010 0000 0000 0000 0000
Sign	Exponent	Mantissa

图5.29 32位浮点版本2

1 bit	8 bits	23 bits
0	00000111	110 0100 0000 0000 0000 0000
Sign	Exponent	Fraction

显而易见，有许多合理的方式来表示浮点数。多年来，计算机制造商使用不兼容的浮点格式。一台计算机的结果无法直接被另一台计算机解读。

电气与电子工程师协会（IEEE）通过于1985年创建IEEE 754 解决了这一问题，IEEE 754 标准定义了浮点数。这种浮点数格式如今几乎被普遍采用，正是本节所讨论的格式。

在二进制浮点数中，尾数的第一位（二进制小数点左侧）始终为1，因此无需存储。该位被称为*implicit leading one*。图5.29展示了 $228_{10} = 11100100_2 \times 2^0 = 1.11001_2 \times 2^7$ 的改进浮点数表示。为提高效率，隐含的前导1不包含在23位尾数中，仅存储小数部分。这释放了一个额外位用于有效数据。

我们对指数字段进行了最终修改。指数需要同时表示正指数和负指数。为此，浮点数使用*biased*指数，即原始指数加上一个常数偏移量。32位浮点数使用的偏移量为127。例如，对于指数7，偏移后的指数为 $7 + 127 = 134 = 10000110_2$ 。对于指数-4，偏移后的指数为： $-4 + 127 = 123 = 01111011_2$ 。图5.30展示了 $1.11001_2 \times 2^7$ 在浮点数表示法中的形式，其中包含隐式前导1和偏移后的指数134（7 + 127）。这种表示法符合IEEE 754 浮点标准。

特殊情况：0、 $\pm\infty$ 和 NaN
IEEE浮点标准包含特殊情形来表示零、无穷大及非法结果等数值。例如，由于隐含的前导1，浮点表示法在表示数字零时存在问题。指数位全为0或全为1的特殊编码被保留用于这些特殊情形。表5.4展示了0、 $\pm\infty$ 和NaN的浮点表示。与符号/数值表示法类似，浮点表示法同时存在正零和负零。NaN用于表示不存在的数值，例如-1或 $\log_2(-5)$ 。



单精度、双精度和四精度格式
到目前为止，我们研究了32位浮点数。这种格式也被称为*single-precision*、*single*或*float*。IEEE 754标准还

图5.30 IEEE 754浮点数表示法

1 bit	8 bits	23 bits
0	10000110	110 0100 0000 0000 0000 0000
Sign	Biased Exponent	Fraction

表5.4 IEEE 754浮点数表示法中的0、±∞和NaN

Number	Sign	Exponent	Fraction
0	X	00000000	000000000000000000000000
∞	0	11111111	000000000000000000000000
−∞	1	11111111	000000000000000000000000
NaN	X	11111111	Non-zero

表5.5 浮点格式

Format	Total Bits	Sign Bits	Exponent Bits	Fraction Bits	Bias
Single	32	1	8	23	127
Double	64	1	11	52	1023
Quad	128	1	15	112	16363

定义64位*double-precision*数（也称为*doubles*）和128位*quadruple-precision*数（也称为*quads*），它们提供更高的精度和更大的范围。表5.5显示了每种格式中字段所使用的位数。

排除前面提到的特殊情况，普通单精度数的范围是±1.175494 × 10^{−38}到±3.402824 × 10³⁸，精度约为7位有效十进制数字（因为2^{−24} ≈ 10^{−7}）。类似地，普通双精度数的范围是±2.22507385850720 × 10^{−308}到±1.797693 13486232 × 10³⁰⁸，精度约为15位有效十进制数字。四精度数具有34位十进制数字的精度，但在硬件或软件中尚未得到广泛支持。

四舍五入
算术结果若超出可用精度范围，必须舍入至邻近数值。舍入模式包括向下舍入、向上舍入、向零舍入以及最近舍入。默认舍入模式为最近舍入。在最近舍入模式下，若两个数值距离相等，则选择分数最低有效位为0的那个数。

回忆一下，当一个数 *overflows* 的数值过大而无法表示时，它会发生溢出。同样地，当一个数 *underflows* 过小以至于无法表示时，它会发生下溢。在舍入到最近模式中，溢出会向上舍入为 ±∞，下溢则会向下舍入为 0。

浮点数无法精确表示某些数字，例如1.7。然而，当你在计算器中输入1.7时，你会看到精确的1.7，而不是1.69999…。为了处理这种情况，一些应用程序（如计算器和金融软件）使用 *binary coded*（BCD）数字或采用以10为底指数的格式。BCD数字使用四位二进制编码每个十进制数字，范围从0到9。例如，1.7的BCD定点表示法（四位整数位和四位小数位）为0001.0111。当然，没有免费的午餐。代价是算术硬件复杂度增加、编码浪费（A–F编码未被使用），从而导致性能下降。因此，对于计算密集型应用，浮点数要快得多。

浮点加法
浮点数加法不像补码加法那样简单。同号浮点数相加的步骤如下：

- 1. 提取指数和小数位。
- 2. 在前面添加前导1以形成尾数。
- 3. 比较指数。
- 4. 必要时移位较小的尾数。
- 5. 相加尾数。
- 6. 规范化尾数并在必要时调整指数。
- 7. 对结果进行舍入。
- 8. 将指数和小数重新组合成浮点数。

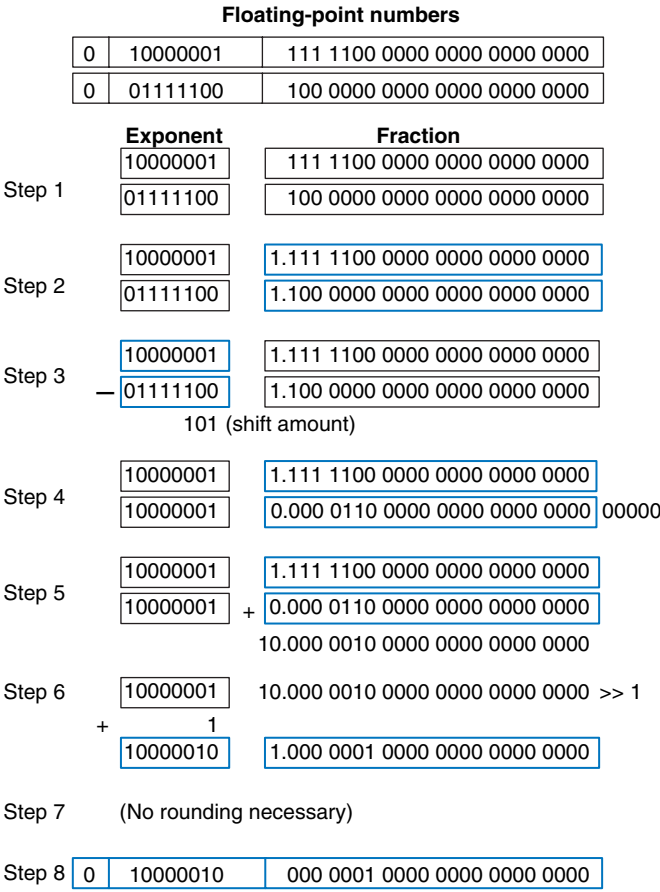


图5.31 浮点数加法

图5.31展示了 $7.875 (1.11111 \times 2^2)$ 与 $0.1875 (1.1 \times 2^{-3})$ 的浮点数加法运算, 结果为 $8.0625 (1.0000001 \times 2^3)$ 。在步骤1和2中提取分数位和指数位, 并前置隐含的前导1后, 通过从较大指数中减去较小指数来比较指数。所得结果即为步骤4中较小数右移以对齐隐含二进制小数点(即令指数相等)所需的位数。对齐后的数值相加。由于和的尾数大于等于2.0, 需通过右移一位并递增指数来归一化结果。本例中结果精确, 因此无需舍入。最终通过移除尾数的隐含前导1并前置符号位, 以浮点表示法存储结果。

浮点运算通常在硬件中执行以提升速度。这种硬件称为 *floating-point unit (FPU)*, 通常与 *central processing unit (CPU)* 相区别。著名的Pentium FPU中的 *floating-point division (FDIV)* 漏洞导致英特尔花费4.75亿美元召回并更换有缺陷的芯片。该漏洞的产生仅仅是因为一个查找表未正确加载。

5.4 顺序构建模块

本节探讨时序构建模块, 包括计数器和移位寄存器。

5.4.1 计数器

如图5.32所示, 一个 N 位 *binary counter* 是一个具有时钟和复位输入以及 N 位输出的时序算术电路, Q . *Reset* 将输出初始化为0。随后, 计数器按二进制顺序遍历所有 2^N 个可能的输出, 并在时钟上升沿递增。

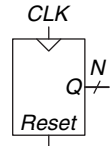


图5.32 计数器符号

图5.33展示了一个由加法器和可复位寄存器组成的 N 位计数器。在每个周期中, 计数器将寄存器中存储的值加1。HDL示例5.4展示了一个具有异步复位的二进制计数器, 图5.34显示了生成的硬件。

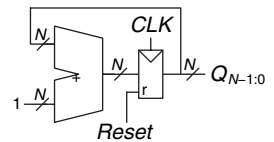


图5.33 N 位计数器

一个 N 位计数器的最高位每 2^N 个周期翻转一次。因此, 它将时钟频率降低了 2^N 倍。这被称为 *divide-by- 2^N* 计数器, 用于减慢快速信号。例如, 如果一个数字系统具有 50 MHz 的内部时钟, 你可以使用一个24位计数器来产生一个 $(50 \times 10^6 \text{ Hz} / 2^{24}) = 2.98 \text{ Hz}$ 的信号, 该信号以人眼可观察到的速率闪烁发光二极管 (LED)。

产生任意频率的另一种计数器泛化称为 *digitally controlled oscillator (DCO, 示例5.7)*。考虑一个 N 位计数器, 每个周期加 p , 而不是加1。如果该计数器接收频率为 f_{clk} 的时钟, 则最高有效位现在以 $f_{out} = f_{clk} \times p / 2^N$ 的频率切换。通过明智地选择 p 和 N , 可以产生任意频率的输出。较大的 N 能以更多硬件为代价提供更精确的控制。

HDL示例5.4 计数器

SystemVerilog

```
module counter #(parameter N = 8)
    (input logic clk,
     input logic reset,
     output logic [N-1:0] q);

    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else q <= q + 1;
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD_UNSIGNED.ALL;

entity counter is
    generic(N: integer := 8);
    port(clk, reset: in STD_LOGIC;
         q: out STD_LOGIC_VECTOR(N-1 downto 0));
end;

architecture synth of counter is
begin
    process(clk, reset) begin
        if reset then q <= (OTHERS => '0');
        elsif rising_edge(clk) then q <= q + '1';
        end if;
    end process;
end;
```

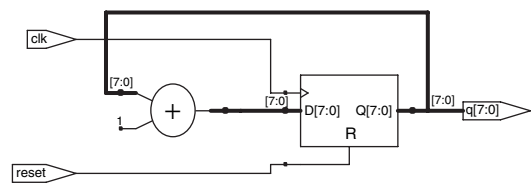


图5.34 合成计数器

示例5.7 数字控制振荡器

假设你有一个50 MHz的时钟，想要产生一个500 Hz的输出。考虑使用一个 $N = 24$ 位或32位的计数器。你应该选择什么 p 值，并且能多接近500 Hz？

解 我们想要 $p/2^N = 500\text{ Hz}/50\text{ MHz} = 0.001$ 。如果 $N = 24$ ，选择 $p = 168$ 得到 $f_{\text{out}} = 500.68\text{ Hz}$ 。如果 $N = 32$ ，选择 $p = 42950$ 得到 $f_{\text{out}} = 500.038\text{ Hz}$ 。

其他类型的计数器，如加/减计数器，将在练习5.51至5.54中探讨。

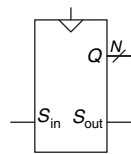


图5.35 移位寄存器符号

5.4.2 移位寄存器

如图5.35所示，*shift register*具有一个时钟、一个串行输入 S_{in} 、一个串行输出 S_{out} 以及 N 个并行输出 $Q_{N-1:0}$ 。在每个时钟上升沿，一个新比特从 S_{in} 移入，所有后续内容向前移位。移位寄存器中的最后一位可在 S_{out} 处获取。

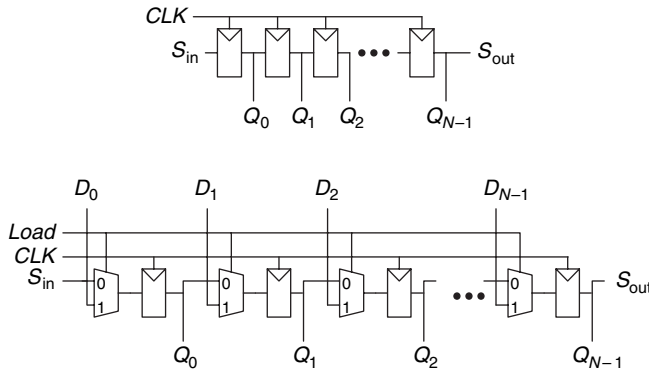


图5.36 移位寄存器 原理图

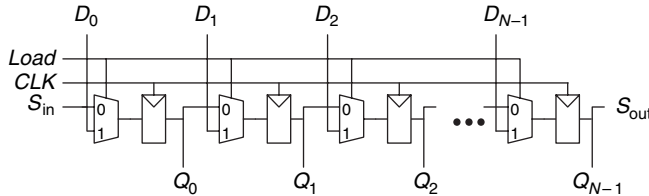


图5.37 带并行加载的移位寄存器

移位寄存器可视为*serial-to-parallel converters*。输入在 S_{in} 处串行提供（一次一位）。经过 N 个周期后，过去的 N 个输入在 Q 处并行可用。

移位寄存器可由 N 个触发器串联构成，如图5.36所示。部分移位寄存器还没有复位信号，用于初始化所有触发器。

一种相关电路是*parallel-to-serial*转换器，它并行加载 N 位数据，然后一次移出一位。通过添加并行输入 $D_{N-1:0}$ 和控制信号 $Load$ ，移位寄存器可以修改为同时执行串行到并行和并行到串行操作，如图5.37所示。当 $Load$ 被断言时，触发器从 D 输入并行加载数据。否则，移位寄存器正常移位。HDL示例5.5描述了这样一个移位寄存器，图5.38显示了生成的硬件。

扫描链*

移位寄存器常用于测试时序电路，采用一种称为*scan chains*的技术。测试组合电路相对直接，只需施加称为*test vectors*的已知输入，并将输出与预期结果进行比对。而测试时序电路则更为困难，因为电路具有状态。从已知初始条件出发，可能需要大量周期的测试向量才能使电路进入期望状态。例如，测试32位计数器的最高位从0跳变为1，需要先复位计数器，再施加 2^{31} （约20亿）个时钟脉冲！

不要将 *shift registers* 与第5.2.5节中的 *shifters* 混淆。移位寄存器是每个时钟边沿移入一个新位的时序逻辑模块。移位器则是将输入按指定量移位的无时钟组合逻辑模块。

为解决此问题，设计者希望能够直接观察并控制机器的所有状态。为此，他们添加了一种测试模式，在该模式下，所有触发器的内容均可被读出或加载所需值。大多数系统拥有过多触发器，无法为每个触发器分配独立引脚进行读写。因此，系统中的所有触发器被连接成一个称为*scan chain*的移位寄存器。

HDL示例5.5 带并行加载的移位寄存器

SystemVerilog

```
module shiftreg #(parameter N = 8)
    (input logic      clk,
     input logic      reset, load,
     input logic      sin,
     input logic [N-1:0] d,
     output logic [N-1:0] q,
     output logic      sout);

    always_ff @(posedge clk, posedge reset)
        if (reset)    q <= 0;
        else if (load) q <= d;
        else          q <= {q[N-2:0], sin};

    assign sout = q[N-1];
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;

entity shiftreg is
    generic(N: integer := 8);
    port(clk, reset: in  STD_LOGIC;
         load, sin: in  STD_LOGIC;
         d: in  STD_LOGIC_VECTOR(N-1 downto 0);
         q: out STD_LOGIC_VECTOR(N-1 downto 0);
         sout: out STD_LOGIC);
end;

architecture synth of shiftreg is
begin
    process(clk, reset) begin
        if reset = '1' then q <= (OTHERS => '0');
        elsif rising_edge(clk) then
            if load then    q <= d;
            else            q <= q(N-2 downto 0) & sin;
            end if;
        end if;
    end process;

    sout <= q(N-1);
end;
```

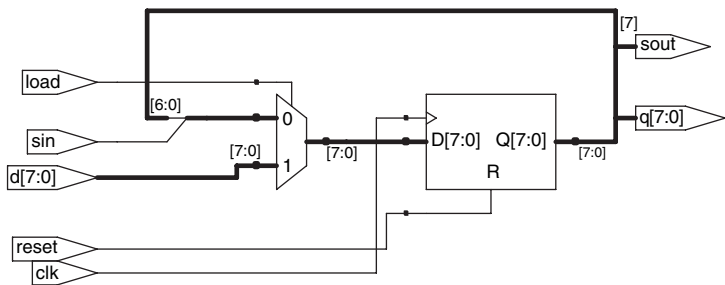


Figure 5.38 Synthesized shiftreg

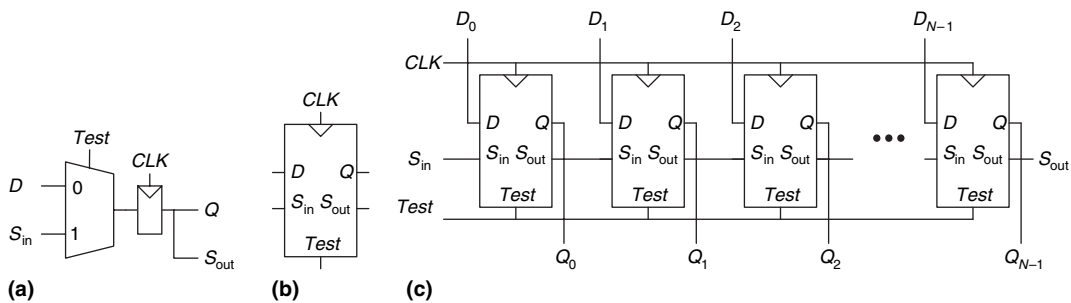


图 5.39 可扫描触发器: (a) 原理图, (b) 符号, (c) N位可扫描 e 寄存器

正常操作时，触发器从其D输入加载数据，并忽略扫描链。在测试模式下，触发器串行移出其内容，并使用 S_{in} 和 S_{out} 移入新内容。加载多路复用器通常集成到触发器中以生成 $scannable\ flip-flop$ 。图5.39展示了可扫描触发器的原理图和符号，并说明了如何级联这些触发器以构建一个N位的可扫描寄存器。

例如，32位计数器可通过在测试模式下移入011111...111模式，在正常模式下计数一个周期，然后移出结果（应为100000...000）进行测试。这仅需 $32 + 1 + 32 = 65$ 个周期。

5.5 存储器阵列

前几节介绍了用于处理数据的算术电路和时序电路。数字系统还需要 *memories* 来存储这些电路使用和生成的数据。由触发器构建的寄存器是一种能够存储少量数据的存储器。本节将介绍能够高效存储大量数据的 *memory arrays*。

我们首先概述所有存储器阵列共有的特性，然后介绍三种类型的存储器阵列：动态随机存取存储器（DRAM）、静态随机存取存储器（SRAM）和只读存储器（ROM）。每种存储器在数据存储方式上有所不同。我们简要讨论面积与延迟的权衡，并展示存储器阵列不仅用于存储数据，还用于执行逻辑功能。最后，通过展示存储器阵列的HDL代码来结束本节。

5.5.1 概述

图5.40展示了一个存储器阵列的通用符号。该存储器被组织成二维的存储单元阵列。存储器对阵列中的某一行进行读取或写入操作，该行由 *address* 指定。读取或写入的值称为 *data*。具有N位地址和M位数据的阵列包含 2^N 行和M列。每一行数据称为一个 *word*。因此，该阵列包含 2^N 个M位字。

图5.41展示了一个具有两个地址位和三个数据位的存储器阵列。两个地址位指定阵列中四行（数据字）中的一行。每个数据字宽度为三位。图5.41(b)显示了该存储器阵列的一些可能内容。

数组的 *depth* 是行数，*width* 是列数，也称为 *word size*。数组的大小表示为 *depth* × *width*。图5.41是一个4字×3位数组，简称为4×3数组。图5.42展示了1024字×32位数组的符号。该数组的总大小为32千比特（Kb），也称为32千比特（Kib）。

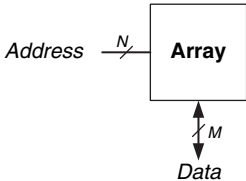
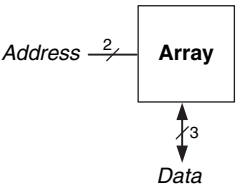


图5.40 通用存储器阵列符号



(a)

Address	Data	
11	0 1 0	depth ↑ ↓
10	1 0 0	
01	1 1 0	
00	0 1 1	
		width ←→

(b)

图5.41 4 × 3 存储器阵列：(a) 符号，(b) 功能

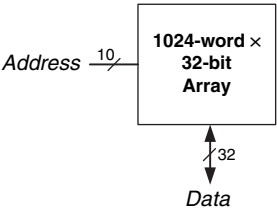


图5.42 32-Kb阵列：深度 = 2^{10} = 1024字，宽度 = 32位

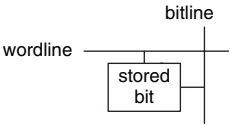


图5.43 位单元

位单元

存储阵列由 *bit cells* 单元构成，每个单元存储1位数据。图5.43显示每个位单元连接至 *wordline* 和 *bitline*。对于每个地址位组合，存储器会激活单条字线，从而激活该行的位单元。当字线为高电平时，存储位通过位线传输或接收数据；否则位线与位单元断开连接。存储位的电路结构因存储器类型而异。

读取位单元时，位线初始处于高阻态（Z）。随后，字线导通，使存储值将位线驱动至0或1。写入位单元时，位线被强驱动至目标值。接着，字线导通，将位线与存储位连接。强驱动的位线会覆盖位单元中的原有内容，从而将目标值写入存储位。

组织

图5.44展示了一个4×3存储阵列的内部结构。当然，实际存储器要大得多，但较大阵列的行为可以从较小阵列中推断出来。在此示例中，该阵列存储了图5.41(b)中的数据。

在内存读取过程中，字线被激活，对应行的位单元将位线驱动为高电平或低电平。在内存写入过程中，位线首先被驱动为高电平或低电平，然后字线被激活，使得位线值能够存储在该行位单元中。例如，要读取 *Address* 10，位线保持浮空，解码器激活 *wordline*₂，该行位单元中存储的数据（100）读出到 *Data* 位线上。要将值001写入

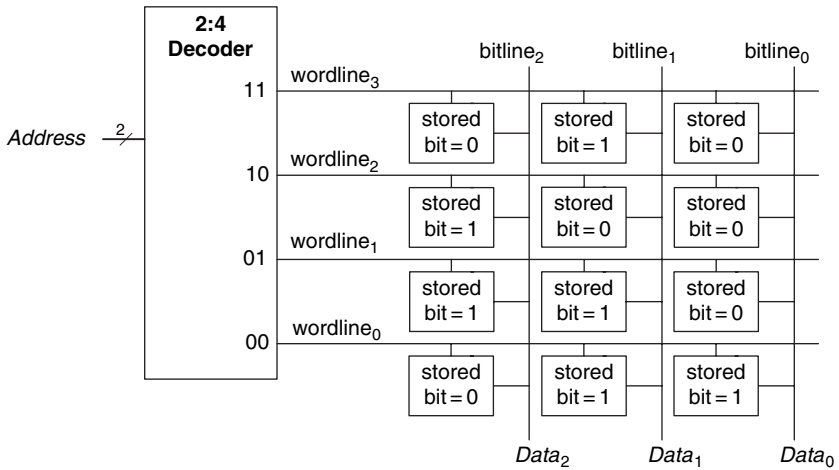


图5.44 4 × 3 存储阵列

Address11, 位线被驱动至值001, 然后wordline₃被置位, 新值(001)被存储到位单元中。

内存端口

所有存储器都有一个或多个ports。每个端口提供对一个存储器地址的读和/或写访问。之前的示例都是单端口存储器。

*Multiported*存储器可以同时访问多个地址。图5.45展示了一个具有两个读端口和一个写端口的三端口存储器。端口1从地址A1读取数据到读数据输出RD1。端口2从地址A2读取数据到RD2。端口3在时钟上升沿将写数据输入WD3的数据写入地址A3, 前提是写使能WE3有效。

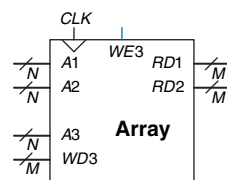


图5.45 三端口存储器

内存类型

存储器阵列由其大小（深度×宽度）以及端口数量和类型来指定。所有存储器阵列都将数据存储为位单元阵列，但它们在存储位的方式上有所不同。

存储器根据其在存储单元中存储位的方式进行分类。最广泛的分类是*random access memory* (RAM) 与*read only memory* (ROM)。RAM是*volatile*, 意味着断电时会丢失数据。ROM是*nonvolatile*, 意味着即使没有电源也能无限期保留数据。

RAM和ROM的名称源于历史原因, 如今已不再具有明确含义。RAM被称为*random*存储器, 因为任何数据字的访问延迟都相同。相比之下, *sequential*存储器（如磁带录音机）访问附近数据的速度比远处数据（例如磁带另一端的数据）更快。ROM被称为*read only*存储器, 因为历史上它只能读取而不能写入。这些名称容易混淆, 因为ROM同样支持随机访问。更糟糕的是, 大多数现代ROM既可写入也可读取! 需要记住的重要区别是: RAM是易失性的, 而ROM是非易失性的。

两种主要的RAM类型是*dynamic RAM* (DRAM) 和*static RAM* (SRAM)。动态RAM将数据以电容上的电荷形式存储, 而静态RAM则使用一对交叉耦合的反相器存储数据。ROM有多种类型, 其写入和擦除方式各不相同。这些不同类型的存储器将在后续章节中讨论。



罗伯特·登纳德, 1932年出生, 1966年在IBM发明了DRAM。尽管许多人怀疑这一想法能否实现, 但到20世纪70年代中期, 几乎所有计算机都采用了DRAM。他自称在进入IBM之前几乎没有进行过创造性工作, 而IBM给了他一本专利笔记本, 并说: “把你所有的想法都记在里面。”自1965年以来, 他在半导体和微电子领域获得了35项专利。(照片由IBM提供。)

5.5.2 动态随机存取存储器 (DRAM)

DRAM, 读作“迪-拉姆”, 通过电容器上电荷的有无来存储一个比特。图5.46展示了一个DRAM比特单元。比特值存储在电容器上。nMOS晶体管充当开关。

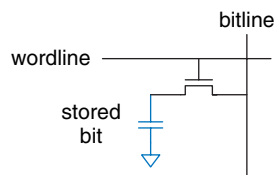
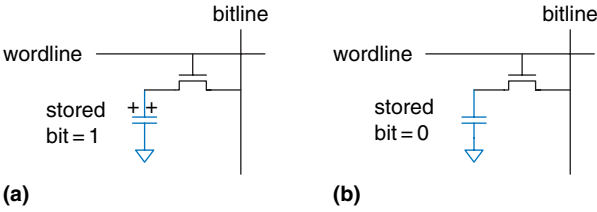


图5.46 DRAM位单元

图5.47 DRAM存储值



该晶体管用于将电容器与位线连接或断开。当字线被激活时，nMOS晶体管导通，存储的比特值便传输到位线或从位线读出。

如图5.47(a)所示，当电容器充电至 V_{DD} 时，存储的位为1；当它放电至GND（图5.47(b)）时，存储的位为0。电容器节点为*dynamic*，因为它没有被连接到 V_{DD} 或GND的晶体管主动驱动为高电平或低电平。

读取时，数据值从电容器传输到位线。写入时，数据值从位线传输到电容器。读取操作会破坏电容器中存储的位值，因此每次读取后必须恢复（重写）数据字。即使DRAM未被读取，其内容也必须每隔几毫秒刷新（读取并重写）一次，因为电容器上的电荷会逐渐泄漏。

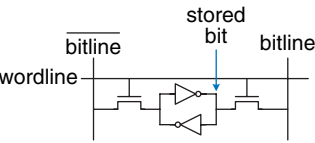


图5.48 SRAM位单元

5.5.3 静态随机存取存储器（SRAM）

SRAM，读作“es-ram”，是*static*，因为存储的位不需要刷新。图5.48展示了一个SRAM位单元。数据位存储在交叉耦合的反相器中，如第3.2节所述。每个单元有两个输出：位线和位线。当字线被断言时，两个nMOS晶体管导通，数据值被传输到位线或从位线传输出来。与DRAM不同，如果噪声降低了存储位的值，交叉耦合的反相器会恢复该值。

5.5.4 面积与延迟

触发器、SRAM和DRAM均为易失性存储器，但各自具有不同的面积和延迟特性。表5.6展示了这三种易失性存储器的对比。触发器存储的数据位可立即从其输出端获取，但构建一个触发器至少需要20个晶体管。通常，器件所含晶体管越多，所需面积、功耗和成本就越高。DRAM的延迟比SRAM更长，因为其位线并非由晶体管主动驱动，必须等待电荷从电容（相对）缓慢地移动到位线。此外，DRAM的吞吐量本质上更低。

表5.6 内存比较

Memory Type	Transistors per Bit Cell	Latency
Flip-flop	~20	Fast
SRAM	6	Medium
DRAM	1	Slow

与SRAM相比，因为它必须定期刷新数据并在读取后刷新。为解决此问题，开发了诸如*synchronous DRAM*（SDRAM）和*double data rate*（DDR）SDRAM等DRAM技术。SDRAM使用时钟对内存访问进行流水线处理。DDR SDRAM（有时简称为DDR）利用时钟的上升沿和下降沿来访问数据，从而在给定时钟速度下使吞吐量翻倍。DDR于2000年首次标准化，运行频率为100至200 MHz。后来的标准DDR2、DDR3和DDR4提高了时钟速度，到2021年速度已超过3 GHz。

内存延迟和吞吐量也取决于内存大小；在其他条件相同的情况下，较大的内存往往比较小的内存更慢。特定设计的最佳内存类型取决于速度、成本和功耗约束。

5.5.5 寄存器文件

数字系统通常使用多个寄存器来存储临时变量。这组寄存器称为*register file*，通常构建为一个小型多端口SRAM阵列，因为它比触发器阵列更紧凑。在某些寄存器文件中，特定条目（如寄存器0）被硬连线为始终读取值0，因为0是常用的常数。

图5.49展示了一个由32个寄存器组成的×32位三端口寄存器文件，该文件基于图5.45的三端口存储器构建。该寄存器文件具有两个读取端口（A1/RD1和A2/RD2）和一个写入端口（A3/WD3）。5位地址——A1、A2和A3——各自可以访问全部 $2^5 = 32$ 个寄存器。因此，可以同时读取两个寄存器并写入一个寄存器。

5.5.6 只读存储器（ROM）

ROM将位存储为晶体管的存在与否。图5.50展示了一个简单的ROM位单元。要读取该单元，位线被弱上拉至高电平。然后，字线被开启。如果存在晶体管，它会将位线拉至低电平。如果不存在，位线保持高电平。

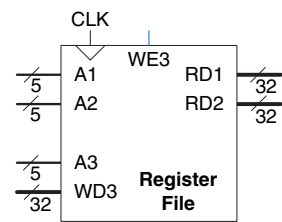


图5.49 32 × 32寄存器文件，具有两个读端口和一个写端口

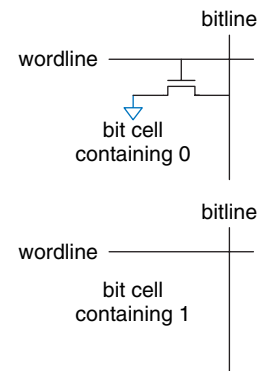


图5.50 包含0和1的ROM位单元

请注意，ROM位单元是一种组合电路，在断电时没有状态可以“遗忘”。

ROM的内容可以用*dot notation*表示。图5.51展示了包含图5.41数据的4字×3位ROM的点表示法。行（字线）与列（位线）交点处的点表示该数据位为1。例如，顶部字线在Data₁上有一个点，因此存储在Address 11处的数据字为010。

从概念上讲，ROM可以通过两级逻辑构建，即一组与门后接一组或门。与门产生所有可能的最小项，因此构成一个译码器。图5.52展示了使用译码器和或门构建的图5.51中的ROM。图5.51中的每个点行对应图5.52中一个或门的输入。对于

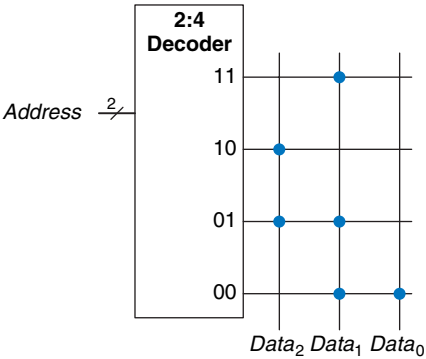


图5.51 4 × 3 ROM：点表示法

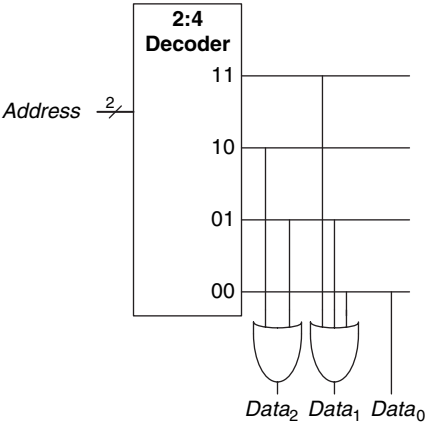


图5.52 4 × 3 使用门电路的ROM实现

数据位带有一个点——在此情况下为 $Data_0$ ——不需要或门。这种ROM的表示方式很有趣，因为它展示了ROM如何执行任意两级逻辑功能。实际上，ROM由晶体管而非逻辑门构建，以减小其尺寸和成本。第5.6.3节将进一步探讨晶体管级实现。

图5.50中ROM位单元的内容在制造过程中通过每个位单元中是否存在晶体管来指定。*programmable ROM* (PROM, 发音类似舞蹈) 在每个位单元中放置一个晶体管，但提供了一种将晶体管连接或断开到地的方式。

图5.53展示了*fuse-programmable ROM*的位单元。用户通过施加高电压选择性地熔断保险丝来对ROM进行编程。如果保险丝存在，晶体管连接到GND，单元存储0；如果保险丝被熔断，晶体管与地断开，单元存储1。这也被称为*one-time programmable ROM*，因为保险丝一旦熔断就无法修复。

可重编程ROM提供了一种可逆机制，用于将晶体管连接或断开至GND。*Erasable PROMs* (EPROMs, 读作“e-proms”) 用 *floating-gate transistor* 替代了nMOS晶体管和熔丝。浮置栅极在物理上不与其他任何导线相连。当施加适当的高电压时，电子通过绝缘体隧穿到浮置栅极上，从而导通晶体管，将位线连接到字线（译码器输出）。当EPROM暴露在强紫外线 (UV) 下约半小时后，电子从浮置栅极上被击出，晶体管关断。这些操作分别称为 *programming* 和 *erasing*。*Electrically erasable PROMs* (EEPROMs, 读作“e-e-proms”或“双e proms”) 和 *Flash* 存储器采用类似原理，但在芯片上集成了擦除和编程电路，因此无需紫外线。EEPROM位单元可单独擦除；闪存则擦除更大的位块，且由于所需擦除电路更少，成本更低。2021年，闪存成本约为每GB 0.10美元，且价格以每年30%至40%的速度持续下降。闪存已成为在相机、音乐播放器等便携式电池供电系统中存储大量数据的极受欢迎的方式。

总之，现代ROM并非真正的只读存储器；它们也可以被编程（写入）。RAM与ROM的区别在于，ROM写入时间较长，但具有非易失性。

5.5.7 使用存储阵列的逻辑

虽然它们主要用于数据存储，但存储器阵列也可以执行组合逻辑功能。例如，图5.51中ROM的 $Data_2$ 输出是两个Address输入的异或结果。同样地，

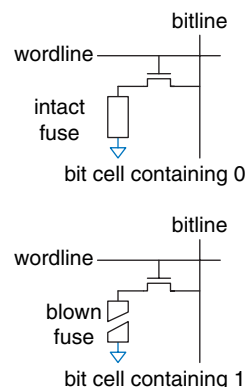


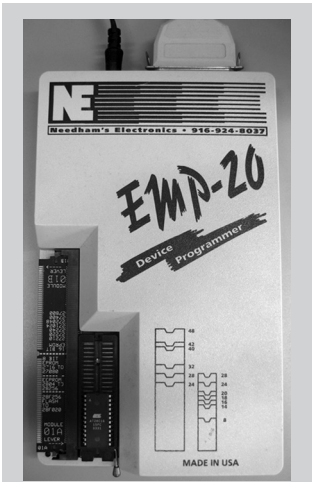
图5.53 熔丝可编程ROM位单元



藤尾增冈，1944年出生，获得日本东北大学电气工程博士学位。1971年至1994年在东芝公司从事存储器与高速电路研发。20世纪70年代末，利用夜间和周末时间以非授权项目形式发明了闪存。闪存得名于其擦除存储器的过程让人联想到相机的闪光灯。东芝对该创意的商业化进程缓慢；英特尔于1988年率先将其推向市场。如今闪存已发展成年产值1700亿美元的市场。舛冈博士后来加入东北大学任教，致力于开发三维晶体管。



采用通用串行总线（USB）接口的闪存驱动器已取代软盘和光盘用于文件共享，因为闪存成本大幅下降。



可编程ROM可通过如下所示的设备编程器进行配置。该设备编程器连接至计算机，由计算机指定ROM类型及待编程的数据值。设备编程器会熔断熔丝或向ROM的浮置栅极注入电荷。因此，这一编程过程有时被称为 *burning* ROM。

$Data_0$ 是两个输入的NOR。一个 $2^N \times M$ 位存储器可以执行任意 N 输入和 M 输出的组合函数。例如，图5.51中的ROM执行两个输入的三项功能。

用于执行逻辑的存储器阵列称为 *lookup tables* (LUTs)。图5.54展示了一个 4×1 位存储器阵列，用作查找表来执行函数 $Y = AB$ 。通过使用存储器执行逻辑，用户可以为给定的输入组合（地址）查找输出值。每个地址对应真值表中的一行，每个数据位对应一个输出值。

5.5.8 内存硬件描述语言

HDL示例5.6描述了一个 $2^N \times M$ 位RAM，图5.55显示了最终的硬件。该RAM具有同步使能写入功能。换句话说，如果写使能 *we* 被置位，则在时钟上升沿进行写入。读取立即发生。首次上电时，RAM的内容是不可预测的。

HDL示例5.7描述了一个 4×3 位ROM。该ROM的内容在HDL的case语句中指定。像这样小的ROM可能会被综合为逻辑门而非阵列。请注意，来自HDL示例4.24的七段译码器在图4.20中被综合为一个ROM。HDL示例5.8展示了一个3端口 32×32 寄存器文件，其中条目0硬连线为0。

5.6 逻辑阵列

与存储器类似，门电路可以组织成规则阵列。如果连接可编程，这些 *logic arrays* 可被配置为

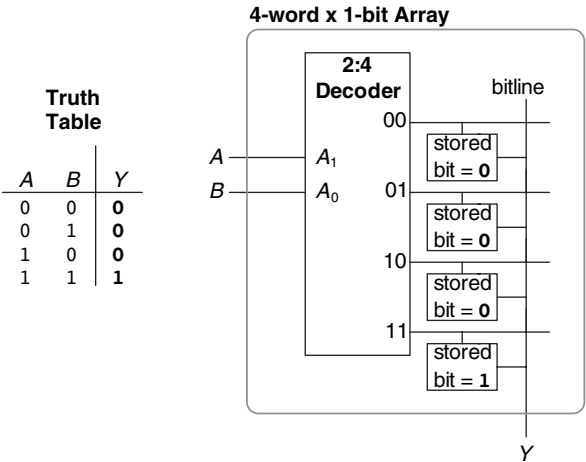


图5.54 用作查找表的 4×1 位存储阵列

HDL示例5.6 RAM

SystemVerilog	VHDL
<pre>module ram #(parameter N = 6, M = 32) (input logic clk, input logic we, input logic [N-1:0] adr, input logic [M-1:0] din, output logic [M-1:0] dout); logic [M-1:0] mem [2**N-1:0]; always_ff @(posedge clk) if (we) mem [adr] <= din; assign dout = mem[adr]; endmodule</pre>	<pre>VHDL library IEEE; use IEEE.STD_LOGIC_1164.ALL; use IEEE.NUMERIC_STD_UNSIGNED.ALL; entity ram_array is generic(N: integer := 6; M: integer := 32); port(clk, we: in STD_LOGIC; adr: in STD_LOGIC_VECTOR(N-1 downto 0); din: in STD_LOGIC_VECTOR(M-1 downto 0); dout: out STD_LOGIC_VECTOR(M-1 downto 0)); end; architecture synth of ram_array is type mem_array is array ((2**N-1) downto 0) of STD_LOGIC_VECTOR (M-1 downto 0); signal mem: mem_array; begin process(clk) begin if rising_edge(clk) then if we then mem(TO_INTEGER(adr)) <= din; end if; end if; end process; dout <= mem(TO_INTEGER(adr)); end;</pre>

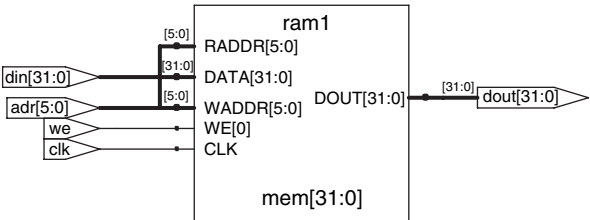


图5.55 合成冲压机

执行任何功能时，用户无需以特定方式连接导线。规则的结构简化了设计。逻辑阵列大规模量产，因此成本低廉。软件工具允许用户将逻辑设计映射到这些阵列上。大多数逻辑阵列还可重新配置，使得无需更换硬件即可修改设计。可重构性在开发过程中很有价值，在现场也很有用，因为只需下载新配置即可升级系统。

本节介绍两种逻辑阵列：可编程逻辑阵列（PLA）和现场可编程门阵列（FPGA）。

HDL示例5.7 只读存储器

SystemVerilog

```

module rom(input  logic [1:0] adr,
           output logic [2:0] dout);

    always_comb
    case(adr)
        2'b00: dout = 3'b011;
        2'b01: dout = 3'b110;
        2'b10: dout = 3'b100;
        2'b11: dout = 3'b010;
    endcase
endmodule

```

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity rom is
    port(adr: in  STD_LOGIC_VECTOR(1 downto 0);
         dout: out STD_LOGIC_VECTOR(2 downto 0));
end;

architecture synth of rom is
begin
    process(all) begin
        case adr is
            when "00" => dout <= "011";
            when "01" => dout <= "110";
            when "10" => dout <= "100";
            when "11" => dout <= "010";
            when others => dout <= "---";
        end case;
    end process;
end;

```

HDL示例5.8 寄存器文件

SystemVerilog

```

module regfile(input  logic      clk,
               input  logic      we3,
               input  logic [5:0] a1, a2, a3,
               input  logic [31:0] wd3,
               output logic [31:0] rd1, rd2);

    logic [31:0] rf[31:0];
    // three ported register file
    // read two ports combinationaly (A1/RD1, A2/RD2)
    // write third port on rising edge of clock (A3/WD3/WE3)
    // register 0 hardwired to 0

    always_ff @(posedge clk)
        if (we3) rf[a3] <= wd3;

    assign rd1 = (a1 != 0) ? rf[a1] : 0;
    assign rd2 = (a2 != 0) ? rf[a2] : 0;
endmodule

```

VHDL

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD_UNSIGNED.all;

entity regfile is
    port(clk: in  STD_LOGIC;
         we3: in  STD_LOGIC;
         a1, a2, a3: in  STD_LOGIC_VECTOR(5 downto 0);
         wd3: in  STD_LOGIC_VECTOR(31 downto 0);
         rd1, rd2: out STD_LOGIC_VECTOR(31 downto 0));
end;

architecture behave of regfile is
    type ramtype is array (31 downto 0) of STD_LOGIC_VECTOR
        (31 downto 0);
    signal mem: ramtype;
begin
    -- three ported register file
    -- read two ports combinationaly (A1/RD1, A2/RD2)
    -- write third port on rising edge of clock (A3/WD3/WE3)
    -- register 0 hardwired to 0
    process(clk) begin
        if rising_edge(clk) then
            if we3 = '1' then mem(to_integer(a3)) <= wd3;
            end if;
        end if;
    end process;
    process(a1, a2) begin
        if (to_integer(a1) = 0) then rd1 <= X"00000000";
        else rd1 <= mem(to_integer(a1));
        end if;
        if (to_integer(a2) = 0) then rd2 <= X"00000000";
        else rd2 <= mem(to_integer(a2));
        end if;
    end process;
end;

```

PLA作为较老的技术，仅执行组合逻辑功能。而FPGA既能执行组合逻辑，也能执行时序逻辑。

5.6.1 可编程逻辑阵列（PLA）

PLA以乘积和（SOP）形式实现两级组合逻辑。PLA由AND阵列后接OR阵列构成，如图5.56所示。输入（以原码和补码形式）驱动AND阵列，生成蕴含项。这些蕴含项随后通过OR运算组合形成输出。一个 $M \times N \times P$ 位的PLA具有 M 个输入、 N 个蕴含项和 P 个输出。

图5.57展示了实现功能 $X = ABC + \overline{A}BC$ 和 $Y = AB$ 的 $3 \times 3 \times 2$ 位PLA的点标记法。AND阵列中的每一行形成一个蕴含项。AND阵列每行中的点表示哪些文字构成该蕴含项。图5.57中的AND阵列形成了三个蕴含项： $\overline{A}BC$ 、 ABC 和 AB 。OR阵列中的点表示哪些蕴含项属于输出函数的一部分。

。

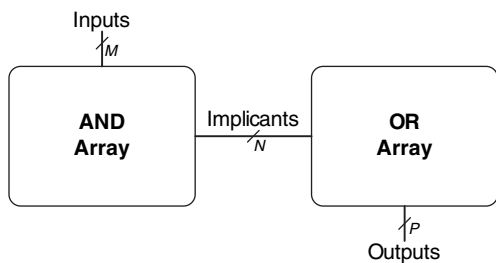


图5.56 $M \times N \times P$ 位PLA

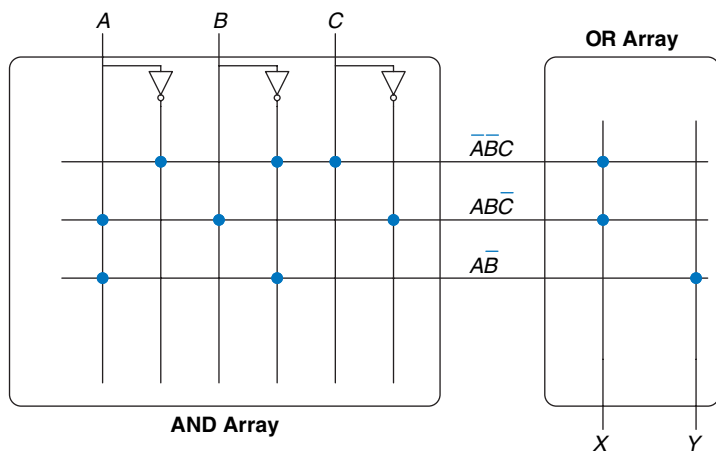


图5.57 $3 \times 3 \times 2$ 位PLA：点表示法

图5.58 3×3×2位PLA 采用两级逻辑

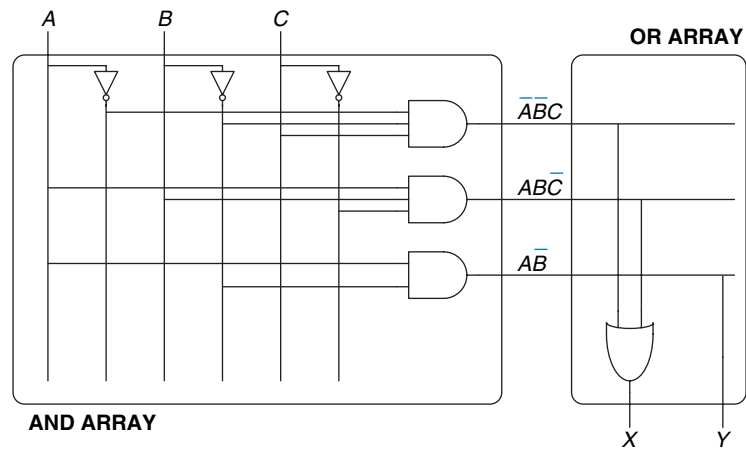


图5.58展示了如何使用两级逻辑构建PLA。另一种实现方式见第5.6.3节。

ROM可视为PLA的特例。一个 2^M 字× N 比特的ROM本质上是一个 M 输入× 2^M 输出× N 比特的PLA。译码器相当于一个AND平面，生成所有 2^M 个最小项。ROM阵列相当于一个OR平面，用于产生输出。如果函数不依赖于所有 2^M 个最小项，则PLA可能比ROM更小。例如，执行图5.57和图5.58中3输入×项×输出2比特PLA所实现的相同功能，需要一个8字×2比特ROM。

Simple programmable logic devices(SPLDs)是增强型的PLA，它在基本的与/或平面上增加了寄存器及其他多种功能。然而，SPLD和PLA在很大程度上已被FPGA取代，因为FPGA在构建大型系统时更加灵活高效。



5.6.2 现场可编程门阵列 (FPGA)

FPGA是一种可重构门阵列。用户通过软件编程工具，可以使用HDL或原理图在FPGA上实现设计。FPGA比PLA更强大、更灵活，原因如下：它们既能实现组合逻辑，也能实现时序逻辑；还能实现多级逻辑函数，而PLA只能实现两级逻辑。现代FPGA集成了其他实用功能，例如内置乘法器、高速I/O、包括模数转换器在内的数据转换器、大容量RAM阵列以及处理器。

FPGA由可配置的logic elements (LEs) 阵列构成，也称为configurable logic blocks (CLBs)。每个LE可配置为执行组合或时序功能。图5.59

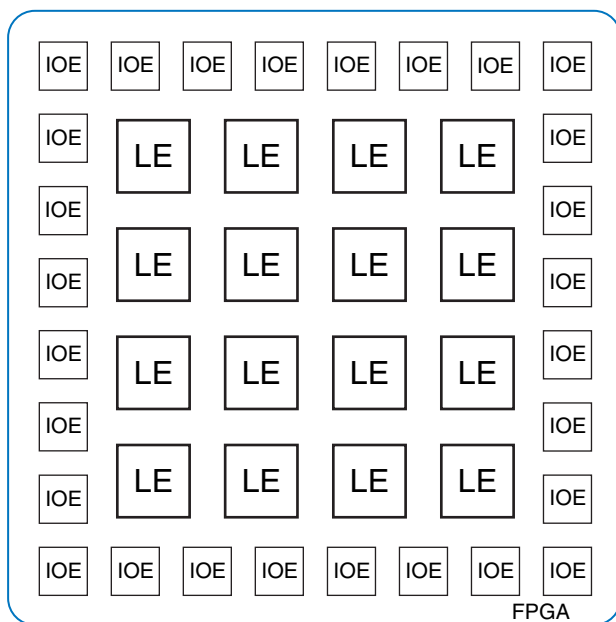


图5.59 通用FPGA布局

展示了FPGA的通用框图。逻辑单元（LE）周围环绕着 *input/output elements*（IOEs），用于与外部世界进行接口。IOE将LE的输入和输出连接到芯片封装的引脚上。LE可以通过可编程路由通道连接到其他LE和IOE。

两家领先的FPGA制造商分别是英特尔（原Altera公司）和赛灵思公司。图5.60展示了英特尔2009年推出的Cyclone IV FPGA中的一个逻辑单元（LE）。该逻辑单元的关键元件是一个4输入查找表（LUT）和一个1位寄存器。逻辑单元还包含可配置的多路选择器，用于在逻辑单元内部路由信号。通过指定查找表的内容以及多路选择器的选择信号，即可完成对FPGA的配置。

每个Cyclone IV逻辑单元（LE）包含一个4输入查找表（LUT）和一个触发器。通过将适当的值加载到LUT中，可将其配置为执行最多四个变量的任意函数。配置FPGA还需选择决定多路复用器如何在LE内部以及向相邻LE和IOE路由数据的选择信号。例如，根据多路复用器的配置，LUT可能从 *data 3* 或LE自身寄存器的输出接收其中一个输入。其他三个输入始终来自 *data 1*、*data 2* 和 *data 4*。*data 1–4* 输入来自IOE或其他LE的输出，具体取决于LE外部的路由。LUT输出要么直接送至

FPGA是许多消费产品的核心，包括汽车、医疗设备以及MP3播放器等媒体设备。例如，梅赛德斯-奔驰S级系列就使用了十多个赛灵思FPGA或PLD，用于从娱乐系统到导航再到巡航控制系统的各种用途。FPGA能够加快产品上市时间，并使得在设计后期进行调试或添加功能变得更加容易。

Altera公司成立于1983年，是最早的FPGA制造商之一，与之并列的还有成立于1984年的Xilinx公司。Altera于2015年被英特尔以167亿美元收购。

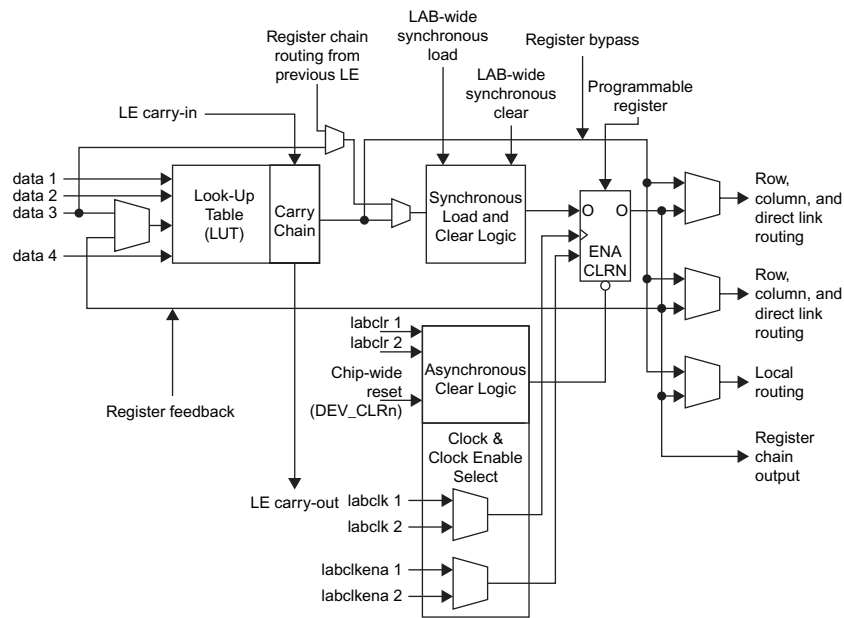


图5.60 Cyclone IV逻辑单元 (LE) (经Altera Cyclone™ IV手册许可转载 © 2010) Altera公司)

组合函数的LE输出，或可通过触发器馈入寄存器函数。触发器的输入来自其自身的LUT输出、data 3输入或前一个LE的寄存器输出。附加硬件包括使用进位链硬件支持加法、用于路由的其他多路复用器，以及触发器的使能和复位功能。Altera将16个LE组合在一起构成一个 *logic array block* (LAB)，并在LAB内的LE之间提供本地连接。

总之，每个 Cyclone IV LE 可执行一个组合和/或寄存器功能，该功能最多可涉及四个变量。其他品牌的 FPGA 组织方式略有不同，但相同的一般原则适用。例如，Xilinx 的 7 系列 FPGA 使用 6 输入 LUT 而非 4 输入 LUT。

设计人员通过首先创建设计的示意图或HDL描述来配置FPGA。然后，该设计被综合到FPGA上。综合工具确定如何配置查找表 (LUT)、多路复用器和路由通道，以执行指定的功能。随后，此配置信息被下载到FPGA中。由于Cyclone IV FPGA将其配置信息存储在SRAM中，因此它们易于重新编程。FPGA可以从实验室的计算机或从

系统启动时，EEPROM芯片会配置FPGA。部分制造商直接在FPGA上集成EEPROM，或使用一次性可编程熔丝来配置FPGA。

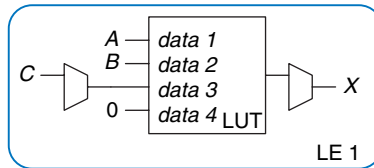
示例5.8 使用LE构建的函数

解释如何配置一个或多个Cyclone IV逻辑单元（LE）以实现以下功能：(a) $X = ABC + \overline{A}B$ 和 $Y = \overline{A}B$ (b) $Y = \overline{A}B$ (c) 一个采用二进制状态编码的三分频计数器（见图3.29(a)）。如有需要，可展示逻辑单元之间的互连。

解 (a) 配置两个逻辑单元（LE）。一个LUT计算X，另一个LUT计算Y，如图5.61所示。对于第一个LE，输入data 1、data 2和data 3分别为A、B和C（这些连接由布线通道设置）。data 4为无关项，但必须连接到某个值，因此将其接0。对于第二个LE，输入data 1和data 2分别为A和B；其他LUT输入为无关项，均接0。配置最终的多路选择器，选择来自LUT的组合逻辑输出，以产生X和Y。通常，单个LE可以以此方式计算最多四个输入变量的任意函数。

(b) 配置第一个LE的LUT以计算 $X = JKLM$ ，并配置第二个LE的LUT以计算 $Y = XPQR$ 。配置最终的多路选择器，从每个LE中选择组合输出X和Y。此配置如图5.62所示。LE之间的路由通道（以蓝色虚线表示）将LE1的输出连接到LE2的输入。通常，一组LE可以以此方式计算N个输入变量的函数。

(A)	(B)	(C)		(X)
data 1	data 2	data 3	data 4	LUT output
0	0	0	X	0
0	0	1	X	1
0	1	0	X	0
0	1	1	X	0
1	0	0	X	0
1	0	1	X	0
1	1	0	X	1
1	1	1	X	0



(A)	(B)	(Y)		
<u>data 1</u>	<u>data 2</u>	<u>data 3</u>	<u>data 4</u>	<u>LUT output</u>
0	0	X	X	0
0	1	X	X	0
1	0	X	X	1
1	1	X	X	0

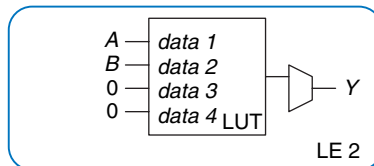


图5.61 用于两个各最多四个输入函数的LE配置

(c) 该FSM具有两位状态 ($S_{1:0}$) 和一位输出 (Y)。下一状态取决于当前状态的两位。使用两个逻辑单元 (LE) 根据当前状态计算下一状态，如图5.63所示。利用两个触发器 (每个LE各一个) 来保持该状态。这些触发器具有复位输入，可连接到外部 *Reset* 信号。寄存器的输出通过 *data 3* 上的多路复用器以及LE之间的路由通道反馈至LUT输入端，如蓝色虚线所示。通常，可能需要另一个LE来计算输出 Y 。然而，在本例中， $Y = S'_0$ ，因此 Y 可来自 LE 1。因此，整个FSM可容纳在两个LE中。通常，一个FSM的每个状态位至少需要一个LE，并且如果输出或下一状态逻辑过于复杂而无法容纳在单个LUT中，则可能需要更多LE。

图5.62 用于超过四个输入的一个函数的LE配置

(J)	(K)	(L)	(M)	(X)	(P)	(Q)	(R)	(X)	(Y)
data 1	data 2	data 3	data 4	LUT output	data 1	data 2	data 3	data 4	LUT output
0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	1	0
0	0	1	0	0	0	0	1	0	0
0	0	1	1	0	0	0	1	1	0
0	1	0	0	0	0	1	0	0	0
0	1	0	1	0	0	1	0	1	0
0	1	1	0	0	0	1	1	0	0
0	1	1	1	0	0	1	1	1	0
1	0	0	0	0	1	0	0	0	0
1	0	0	1	0	1	0	0	1	0
1	0	1	0	0	1	0	1	0	0
1	0	1	1	0	1	0	1	1	0
1	1	0	0	0	1	1	0	0	0
1	1	0	1	0	1	1	0	1	0
1	1	1	0	0	1	1	1	0	0
1	1	1	1	0	1	1	1	1	0
1	1	1	1	1	1	1	1	1	1

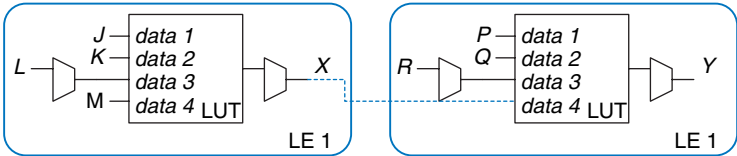
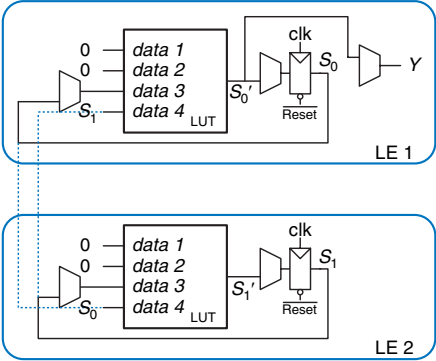


图5.63 具有两位状态的FSM的LE配置

data 1	data 2	(S_0)	(S_1)	(S'_0)
data 3	data 4	LUT output		
X	X	0	0	1
X	X	0	1	0
X	X	1	0	0
X	X	1	1	0

data 1	data 2	(S_1)	(S_0)	(S'_1)
data 3	data 4	LUT output		
X	X	0	0	0
X	X	0	1	1
X	X	1	0	0
X	X	1	1	0



示例5.9 更多逻辑元件示例

构建以下每个电路需要多少个Cyclone IV LE?

- (a) 4输入与门 (b) 7输入异或门 (c) $Y = A(B + C + D + E) + A(BCDE)$ (d) 12位移位寄存器 (e) 32位2:1多路选择器 (f) 16位计数器 (g) 具有2位状态、2个输入和3个输出的任意有限状态机
-

解决方案

- (a) 1: LUT可以执行最多4个输入的任何函数。
- (b) 2: 第一个LUT可以计算一个4输入异或。第二个LUT可以将该输出与另外三个输入进行异或。
- (c) 3: 一个LUT计算 $B + C + D + E$ ，这是一个4输入的函数。A第二个LUT计算 $BCDE$ ，这是另一个4输入的函数。第三个LUT使用3个输入（这两个输出和A）来计算Y。
- (d) 12: 移位寄存器每个比特包含一个触发器。
- (e) 32: 一个2:1多路选择器是三个输入S、 D_0 和 D_1 的函数，因此每个比特需要一个LUT。
- (f) 16: 计数器的每一位需要一个触发器和一位全加器。逻辑单元（LE）包含触发器和加法器逻辑。尽管全加器有两个输出，看起来可能需要两个查找表（LUT），但LE包含如图5.60所示的专用进位链逻辑，经过优化后可用单个LE执行加法运算。
- (g) 5: 该有限状态机有两个触发器、两个次态信号和三个输出信号。每个次态信号是四个变量（两位状态和两个输入）的函数，因此可以用一个查找表计算。因此，两个逻辑单元足以实现次态逻辑和状态寄存器。每个输出最多是四个信号的函数，因此每个输出需要额外一个查找表。
-

示例5.10 延迟

Alyssa P. Hacker 正在构建一个必须以 200 MHz 运行的有限状态机。她使用 Cyclone IV FPGA，其规格如下： t_{LE} = 每个 LE 381 ps， t_{setup} = 76 ps，以及 t_{pcq} = 所有触发器的 199 ps。LE 之间的布线延迟为 246 ps。假设触发器的保持时间为 0。

她的设计在两个寄存器之间可以使用的LE的最大层数是多少？

Alyssa 使用公式 3.13 求解逻辑的最大传播延迟： $t_{pd} \leq T_c - (t_{pcq} + t_{setup})$ 。

因此， $t_{pd} = 5\text{ ns} - (0.199\text{ ns} + 0.076\text{ ns})$ ，所以 $t_{pd} \leq 4.725\text{ ns}$ 。每个LE的延迟加上LE之间的布线延迟 $t_{LE+wire}$ 为 $381\text{ ps} + 246\text{ ps} = 627\text{ ps}$ 。LE的最大数量 N 为 $Nt_{LE+wire} \leq 4.725\text{ ns}$ 。因此， $N = 7$ 。

许多ROM和PLA使用 *dynamic circuits* 替代伪nMOS电路。动态门仅在部分时间内导通pMOS晶体管，当pMOS关闭且不需要结果时节省功耗。除此之外，动态和伪nMOS存储阵列在设计和行为上相似。

5.6.3 数组实现*

为减小尺寸和成本，ROM和PLA通常采用伪nMOS（见第1.7.8节）或动态电路，而非传统逻辑门。

图5.64(a)展示了一个 4×3 位ROM的点标记法，该ROM执行以下功能： $X = A \oplus B$ $Y = A + B$ 、 $Z = AB$ 。这些功能与图5.51中的功能相同，只是地址输入重命名为A和B，数据输出重命名为X、Y和Z。伪nMOS实现如图5.64(b)所示。每个解码器输出连接到其行中nMOS晶体管的栅极。请记住，在伪nMOS电路中，弱pMOS晶体管将输出拉高至 *only if*，前提是通过下拉（nMOS）网络没有到GND的路径。

下拉晶体管放置在每个无点的连接处。图5.64(a)点符号图中的点在图5.64(b)中保留可见，以便于比较。弱上拉晶体管将每个没有下拉晶体管的字线输出拉高。例如，当 $AB = 11$ 时，11字线为高电平，且X和Z上的晶体管

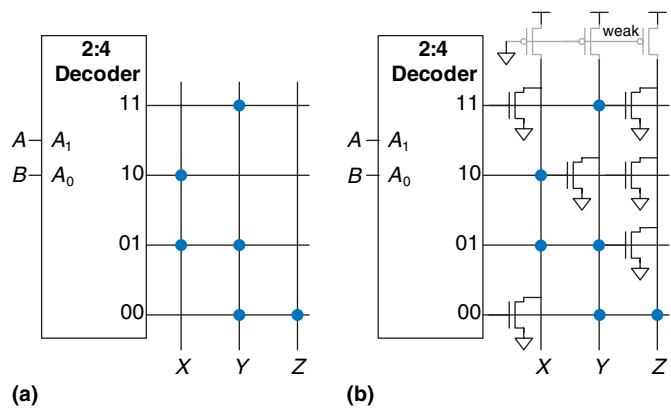


图5.64 ROM实现：(a) 点表示法，(b) 伪nMOS电路

t

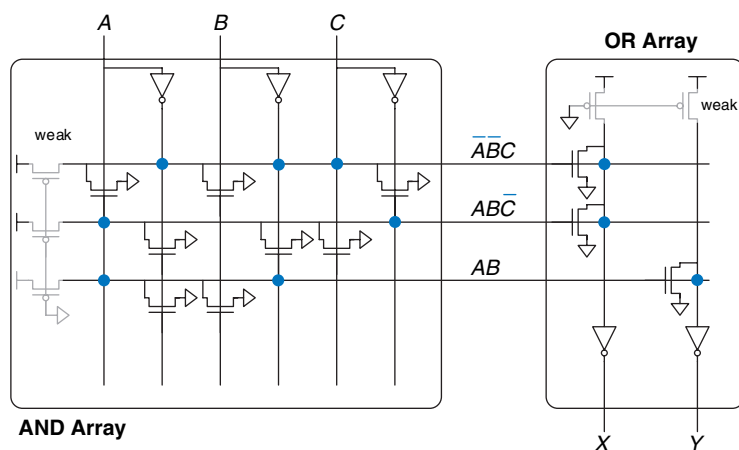


图5.65 $3 \times 3 \times 2$ 位PLA 采用伪nMOS电路

开启并将这些输出拉低。Y输出没有晶体管连接到11字线，因此Y被弱上拉电阻拉高。

PLA也可以使用伪nMOS电路构建，如图5.65所示，该图对应图5.57中的PLA。下拉（nMOS）晶体管放置在AND阵列中带点文字项的 *complement* 上，以及OR阵列中带点的行上。OR阵列中的列在馈送到输出位之前先经过反相器。同样，图5.57点标记图中的蓝点在图5.65中保留可见，以便于比较。

5.7 总结

本章介绍了许多数字系统中使用的数字构建模块。这些模块包括算术电路，如加法器、减法器、比较器、移位器、乘法器和除法器；时序电路，如计数器和移位寄存器；以及用于存储和逻辑的阵列。本章还探讨了定点数和浮点数表示法。在第7章中，我们将使用这些构建模块来构建一个微处理器。

加法器构成了大多数算术电路的基础。半加器将两个1位输入A和B相加，产生一个和与一个进位输出。全加器在半加器的基础上扩展，还接受一个进位输入。N个全加器可以级联形成进位传播加法器（CPA），用于将两个N位数相加。这种类型的CPA称为行波进位加法器，因为进位会依次通过每个全加器。更快的CPA可以通过超前进位或前缀技术构建。

减法器将第二个输入取反后与第一个输入相加。幅度比较器从一个数中减去另一个数，并根据结果的符号或进位输出来确定相对值。

A 乘法器使用与门形成部分积，然后使用全加器对这些位求和。除法器反复从部分余数中减去除数，并检查差值的符号以确定商位。计数器使用加法器和寄存器来递增运行计数。

小数使用定点或浮点形式表示。定点数类似于十进制数，而浮点数则类似于科学记数法。定点数使用普通算术电路，而浮点数需要更复杂的硬件来提取和处理符号、指数和尾数。

大容量存储器被组织成字阵列。存储器具有一个或多个端口用于读取和/或写入字。易失性存储器（如SRAM和DRAM）在断电时会丢失其状态。SRAM比DRAM更快，但需要更多的晶体管。寄存器文件是一种小型多端口SRAM阵列。非易失性存储器（称为ROM）可无限期保持其状态。尽管名称如此，大多数现代ROM是可写入的。

数组也是构建逻辑的常用方式。存储器阵列可用作查找表（LUT）来实现组合逻辑。可编程逻辑阵列（PLA）由可配置的与阵列和或阵列之间的专用连接组成，仅实现组合逻辑。现场可编程门阵列（FPGA）由许多小型查找表和寄存器组成，可实现组合逻辑和时序逻辑。查找表的内容及其互连可配置以实现任意逻辑功能。现代FPGA易于重新编程，且规模足够大、成本足够低，足以构建高度复杂的数字系统，因此广泛应用于中低批量商业产品以及教育领域。

练习

练习5.1 以下类型的64位加法器的延迟是多少？假设每个二输入门延迟为150 ps，全加器延迟为450 ps。

- (a) 一个行波进位加法器
- (b) 一个具有4位块的先行进位加法器
- (c) 前缀加法器

练习5.2 设计两个加法器：一个64位行波进位加法器和一个4位块64位超前进位加法器。仅使用二输入门。每个二输入门面积为 $15\text{ }\mu\text{m}^2$ ，延迟为50 ps，总栅电容为20 fF。假设静态功耗可忽略不计。

- (a) 比较加法器（在100 MHz和1.2 V下工作）的面积、延迟和功耗。（b）讨论功耗、面积和延迟之间的权衡。

练习5.3 解释为什么设计者可能会选择使用纹波进位加法器而不是超前进位加法器。

练习5.4 用HDL设计图5.7中的16位前缀加法器。仿真并测试你的模块以证明其功能正确。

练习5.5 图5.7所示的前缀网络使用黑色单元计算所有前缀。其中一些块传播信号实际上并非必要。设计一种“灰色单元”，它接收位 $i:k$ 和 $k-1:j$ 的G和P信号，但仅生成 $G_{i:j}$ ，而不生成 $P_{i:j}$ 。重新绘制前缀网络，尽可能用灰色单元替换黑色单元。

练习5.6 图5.7所示的前缀网络并非在对数时间内计算所有前缀的唯一方法。Kogge-Stone网络是另一种常见的前缀网络，它通过不同的黑单元连接实现相同功能。研究Kogge-Stone加法器，并绘制类似于图5.7的示意图，展示Kogge-Stone加法器中黑单元的连接方式。

练习5.7 回忆一下，一个N输入优先编码器有 $\log_2 N$ 个输出，用于编码N个输入中哪一个获得优先权（参见练习2.36）。

- (a) 设计一个具有N个输入的优先级编码器，其延迟随N呈对数增长。画出你的设计草图，并给出电路延迟（以电路元件的延迟表示）。

- (b) 用HDL编写你的设计。仿真并测试你的模块以证明其功能正确。

练习5.8 设计以下用于32位无符号数的比较器。绘制原理图。

(a) 不等于 (b) 大于或等于 (c)
) 小于

练习5.9 考虑图5.12中的有符号比较器。

(a) 给出两个4位有符号数 A 和 B 的示例，使得4位有符号比较器能正确计算 $A < B$ 。 (b) 给出两个4位有符号数 A 和 B 的示例，使得4位有符号比较器错误地计算 $A < B$ 。 (c) 一般情况下， N 位有符号比较器在何时会错误运行？

练习5.10 修改图5.12中的 N 位有符号比较器，使其能够正确计算所有 N 位有符号输入 A 和 B 的 $A < B$ 。

练习5.11 使用你喜欢的HDL设计图5.15所示的32位ALU。你可以将顶层模块设计为行为级或结构级。

练习5.12 使用你喜欢的HDL设计图5.17所示的32位ALU。你可以将顶层模块设计为行为级或结构级。

练习5.13 使用你喜欢的HDL设计图5.18(a)中所示的32位ALU。你可以将顶层模块设计为行为级或结构级。

练习5.14 使用你喜欢的HDL设计图5.18(b)所示的32位ALU。你可以将顶层模块设计为行为级或结构级。

练习5.15 编写一个测试平台，用于测试练习5.11中的32位ALU。然后，使用该测试平台对ALU进行测试。包含所有必要的测试向量文件。确保测试足够的边界情况，以说服合理的怀疑者该ALU功能正确。

练习5.16 对练习5.12中的ALU重复练习5.15。

练习5.17 对练习5.13中的ALU重复练习5.15。

练习5.18 对练习5.14中的ALU重复练习5.15。

练习5.19 构建一个比较两个无符号数 A 和 B 的 *unsigned comparison unit*。该单元的输入是来自ALU的 $Flags$ 信号 (N 、 Z 、 C 、 V)

图5.16中，ALU执行减法运算： $A - B$ 。该单元的输出为 HS 、 LS 、 HI 和 LO ，分别表示 A 高于或等于（ HS ）、低于或等于（ LS ）、高于（ HI ）或低于（ LO ） B 。

(a) 写出用 N 、 Z 、 C 和 V 表示 HS 、 LS 、 HI 和 LO 的最小方程。

(b) 绘制 HS 、 LS 、 HI 和 LO 的电路图。

练习5.20 构建一个 *signed comparison unit*，用于比较两个有符号数 A 和 B 。该单元的输入是来自图5.16中ALU的 $Flags$ 信号（ N 、 Z 、 C 、 V ），此时ALU执行减法运算： $A - B$ 。该单元的输出为 GE 、 LE 、 GT 和 LT ，分别表示 A 大于等于（ GE ）、小于等于（ LE ）、大于（ GT ）或小于（ LT ） B 。

(a) 写出用 N 、 Z 、 C 和 V 表示 GE 、 LE 、 GT 和 LT 的最小方程。 (b) 画出 GE 、 LE 、 GT 和 LT 的电路图。

练习5.21 设计一个移位器，始终将32位输入左移2位。输入和输出均为32位。用文字解释该设计并绘制原理图。用你喜欢的HDL实现该设计。

练习5.22 设计4位左旋转器和右旋转器。绘制每个设计的原理图。用你喜欢的HDL实现你的设计。

练习5.23 仅使用24个2:1多路复用器设计一个8位左移位器。该移位器接受一个8位输入 A 和一个3位移位量 $shamt_{2:0}$ ，产生一个8位输出 Y 。画出原理图。

练习5.24 解释如何仅使用 $N \log_2 N$ 个2:1多路复用器构建任意 N 位移位器或循环移位器。

练习5.25 图5.66中的 *funnel shifter* 可以执行任意 N 位的移位或旋转操作。它将 $2N$ 位的输入右移 k 位。输出 Y 是结果的 N 个最低有效位。输入的最高 N 位称为 B ，最低 N 位称为 C 。通过选择合适的 B 、 C 和 k 值，漏斗移位器可以执行任何类型的移位或旋转操作。请根据 A 、 $shamt$ 和 N 解释这些值应如何选取。

(a) A 逻辑右移 $shamt$

(b) A 算术右移 $shamt$

(c) 将 A 左移 $shamt$ 位

(d) 将 A 向右旋转 $shamt$

(e) 将 A 左移 $shamt$ 位

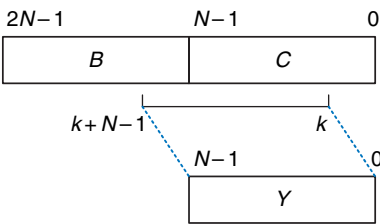


图5.66 漏斗移位器

练习5.26 根据图5.21中的无符号4×4乘法器，以与门延迟(t_{AND})和全加器延迟(t_{FA})表示其关键路径。以相同方式构建的 $N \times N$ 乘法器的延迟是多少？

练习5.27 根据图5.22，以2:1多路选择器延迟(t_{MUX})、加法器延迟(t_{FA})和反相器延迟(t_{INV})表示无符号4 × 4除法器的关键路径。以相同方式构建的 $N \times N$ 除法器的延迟是多少？

练习5.28 设计一个处理二进制补码数的乘法器 数字。

练习5.29 符号扩展单元通过将输入的最高有效位复制到输出的高位，将 M 位的二进制补码数扩展为 N 位 ($N > M$ 位) (参见第1.4.6节)。它接收一个 M 位的输入 A ，并产生一个 N 位的输出 Y 。请设计一个4位输入、8位输出的符号扩展单元电路，并编写该设计的HDL代码。

练习5.30 零扩展单元通过将输出高位补零，将无符号数从 M 位扩展至 N 位 ($N > M$)。设计一个4位输入、8位输出的零扩展单元电路，并绘制其原理图。为该设计编写硬件描述语言代码。

练习5.31 使用小学标准除法算法，计算二进制数111001.000₂/001100.000₂。展示你的计算过程。

练习5.32 以下数字系统可以表示的数字范围是多少？

- (a) U12.12格式 (24位无符号定点数，12位整数位和12位小数位)
- (b) 24位符号/幅值定点数，12位整数位和12位小数位
- (c) Q12.12格式 (24位二进制补码定点数，12位整数位和12位小数位)

练习5.33 将以下十进制数表示为16位定点符号/数值格式，其中包含八位整数位和八位小数位。用十六进制表示你的答案。

(a) -13.5625 (b)
42.3125 (c) $-17.$
15625

练习5.34 将以下十进制数表示为12位定点符号/幅度格式，其中六位整数位和六位小数位。用十六进制表示你的答案。

(a) -30.5 (b) 16.
25 (c) -8.07812
5

练习5.35 将练习5.33中的十进制数表示为Q8.8格式（16位定点二进制补码格式，八位整数位和八位小数位）。用十六进制表示答案。

练习5.36 将练习5.34中的十进制数表示为Q6.6格式
(12位定点二进制补码格式，包含六位整数位和六位小数位)。请用十六进制表示你的答案。

练习5.37 将练习5.33中的十进制数表示为IEEE 754单精度浮点格式。用十六进制表示你的答案。

练习5.38 将练习5.34中的十进制数表示为IEEE 754单精度浮点格式，并以十六进制给出答案。

练习5.39 将以下Q4.4（二进制补码定点数）转换为十进制。隐含的二进制小数点已明确标出，以帮助理解。

(a) 0101.1000 (b)
) 1111.1111 (c)
1000.0000

练习5.40 对以下Q6.5格式（二进制补码定点）数重复练习5.39。

(a) 011101.10101 (b)
) 100110.11010 (c) 1
01000.00100

练习5.41 在将两个浮点数相加时，指数较小的数会被移位。这是为什么？请用文字解释并给出一个例子来证明你的解释。

练习5.42 将以下IEEE 754单精度浮点数相加。

(a) C0123456 + 81C564B7 (b)
D0B10301 + D1B43203 (c) 5
EF10324 + 5E039020

练习5.43 将以下IEEE 754单精度浮点数相加。

(a) C0D20004 + 72407020 (b) C0D20004 + 40DC0004 (c) (5FBE4000 + 3FF80000)
+ DFDE4000 (为什么结果违反直觉？请解释。)

练习5.44 将第5.3.2节中执行浮点数加法的步骤扩展，使其适用于负浮点数以及正浮点数。

练习5.45 考虑IEEE 754单精度浮点数。

(a) IEEE 754单精度浮点格式可以表示多少个数？不需要计算 $\pm\infty$ 或NaN。 (b) 如果不表示 $\pm\infty$ 和NaN，可以额外表示多少个数？ (c) 解释为什么 $\pm\infty$ 和NaN被赋予特殊表示。

练习5.46 考虑以下十进制数：245和0.0625。

(a) 使用单精度浮点表示法写出这两个数，并以十六进制给出答案。 (b) 对(a)部分中的两个32位数进行数值比较。换句话说，将这两个32位数解释为二进制补码数并进行比较。整数比较是否给出了正确的结果？ (c) 你决定提出一种新的单精度浮点表示法。除了使用二进制补码表示指数外，其他所有内容均与IEEE 754单精度浮点标准相同。

而不是偏置。使用你的新标准写出这两个数字。用十六进制给出你的答案。

(d) 整数比较是否适用于你在(c)部分中新的浮点数表示法？

(e) 为什么整数比较使用浮点数更方便？

练习5.47 使用你最喜欢的硬件描述语言设计一个单精度浮点加法器。在用HDL编写设计之前，先画出设计的原理图。对加法器进行仿真和测试，以向怀疑者证明其功能正确。你可以只考虑正数，并使用向零舍入（截断）。你也可以忽略表5.4中给出的特殊情况。

练习5.48 在本问题中，你将探索32位浮点乘法器的设计。该乘法器有两个32位浮点输入，并产生一个32位浮点输出。你只需考虑正数，并使用向零舍入（截断）。同时可以忽略表5.4中给出的特殊情况。

(a) 写出执行32位浮点数乘法所需的步骤。

(b) 绘制一个32位浮点乘法器的原理图。

(c) 使用硬件描述语言设计一个32位浮点乘法器。通过仿真和测试证明该乘法器功能正确，以打消怀疑者的疑虑。

练习5.49 在本问题中，你将探索32位前缀加法器的设计。

(a) 绘制设计示意图。

(b) 用HDL设计32位前缀加法器。仿真并测试你的加法器，以证明其功能正确。

(c) 你从(a)部分设计的32位前缀加法器的延迟是多少？假设每个二输入门延迟为100 ps。

(d) 设计一个32位前缀加法器的流水线版本。绘制出你设计的原理图。你的流水线前缀加法器能运行多快？你可以假设时序开销 ($t_{pcq} + t_{\text{setup}}$) 为80皮秒。使该设计尽可能快地运行。

(e) 使用硬件描述语言设计流水线式32位前缀加法器。

练习5.50 一个增量器将一个N位数字加1。使用半加器构建一个8位增量器。

练习5.51 构建一个32位同步计数器 $Up/Down$ counter。输入为 $Reset$ 和 Up 。当 $Reset$ 为1时,输出全部为0。否则,当 $Up=1$ 时,电路向上计数,当 $Up=0$ 时,电路向下计数。

练习5.52 设计一个在每个时钟边沿加4的32位计数器。该计数器具有复位和时钟输入。复位时,计数器输出全为0。

练习5.53 修改练习5.52中的计数器,使其在每个时钟边沿根据控制信号 $Load$ 要么递增4,要么加载一个新的32位值 D 。当 $Load=1$ 时,计数器加载新值 D 。否则,它递增4。

练习5.54 一个 N 位 $Johnson$ counter由一个带有复位信号的 N 位移位寄存器组成。移位寄存器的输出(S_{out})被取反后反馈到输入端(S_{in})。当计数器复位时,所有位都被清零为0。

(a) 展示4位约翰逊计数器在复位后立即产生的输出序列 $Q_{3:0}$ 。

(b) 一个 N 位约翰逊计数器需要经过多少个周期才会重复其序列? 请解释。

(c) 使用5位约翰逊计数器、十个与门和反相器设计一个十进制计数器。该十进制计数器具有时钟、复位和十个独热输出 $Y_{9:0}$ 。当计数器复位时, Y_0 被置位。在随后的每个周期中,下一个输出应被置位。经过十个周期后,计数器应重复。绘制该十进制计数器的原理图。

(d) 约翰逊计数器相比传统计数器可能具有哪些优势?

练习5.55 编写一个4位可扫描触发器的HDL代码,如图5.39所示。仿真并测试你的HDL模块以证明其功能正确。

练习5.56 英语语言具有相当多的冗余,这使我们能够重建被干扰的传输信息。二进制数据也可以以冗余形式传输,以便进行纠错。例如,数字0可以编码为00000,数字1可以编码为11111。然后,该值可以通过一个可能翻转最多两个比特的噪声信道发送。接收方可以重建原始数据,因为0在接收到的五个比特中至少有三个是0;同样,1至少有三个1。

(a) 提出一种编码方式,使用五比特信息发送00、01、10或11,使得所有破坏编码数据中一比特的错误

可以纠正。提示：分别用于00和11的编码00000和11111将不起作用。

- (b) 设计一个电路，接收你的五位编码数据并将其解码为00、01、10或11，即使传输数据中的一位发生了改变。
- (c) 假设你想改用另一种5位编码。你该如何设计实现，以便在不使用不同硬件的情况下轻松更改编码？

练习5.57 闪存EEPROM，简称闪存，是一项相对较新的发明，彻底改变了消费电子产品。研究并解释闪存的工作原理。使用图示说明浮置栅极。描述存储器中的一位是如何编程的。正确引用你的资料来源。

练习5.58 外星生命项目团队刚刚在莫诺湖底部发现了外星生物。他们需要根据NASA探测器提供的测量特征（绿色度、棕色度、粘滑度和丑陋度）构建一个电路，以按潜在起源星球对这些外星生物进行分类。经过与外星生物学家的仔细磋商，得出以下结论：

- 如果外星人是绿色黏滑的，或者丑陋、棕色且黏滑的，它可能来自火星。
- 如果这个生物丑陋、棕色且黏糊糊的，或者绿色且既不丑陋也不黏糊糊，它可能来自金星。
- 如果这个小怪物是棕色的，既不丑也不黏糊，或者是绿色且黏糊的，它可能来自木星。

请注意，这并非一门精确科学；例如，一种带有绿色和棕色斑点、黏滑但不丑陋的生命形式，可能来自火星或木星。

- (a) 编写一个 $4 \times 4 \times 3$ 的PLA程序来识别外星人。你可以使用点标记法。
- (b) 编程一个 16×3 ROM以识别外星人。你可以使用点表示法。
- (c) 用硬件描述语言实现你的设计。

练习5.59 使用单个 16×3 ROM实现以下函数。使用点符号表示ROM内容。

(a) $X = AB + BCD + A\bar{B}\bar{C}$

b) $Y = AB + BD$ (c)

$Z = A + B + C + D$

练习5.60 使用 $4 \times 8 \times 3$ PLA实现练习5.59中的函数。你可以使用点表示法。

练习5.61 请指定可用于编程以下每个组合电路的ROM的尺寸。使用ROM来实现这些功能是否是一个好的设计选择？请解释原因。

- (a) 一个带有 C_{in} 和 C_{out} 的16位加法器/减法器 (b)
一个 8×8 乘法器 (c) 一个16位优先编码器（参见练习2.36）

练习5.62 考虑图5.67中的ROM电路。对于每一行，能否通过适当编程列II中的ROM，将列I中的电路替换为等效电路？

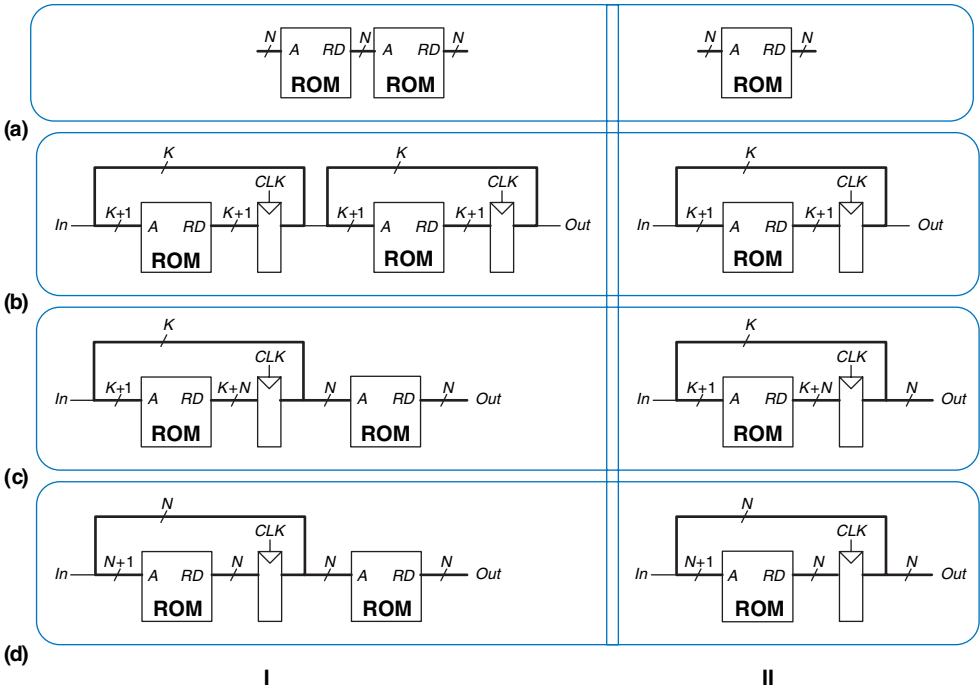


图5.67 ROM电路

练习5.63 实现以下每个功能需要多少个Cyclone IV FPGA逻辑单元（LE）？说明如何配置一个或多个LE来执行这些功能。

函数。您应能通过检查完成此操作，无需执行逻辑综合。

- (a) 练习2.13(c)中的组合函数 (b) 练习2.17(c)中的组合函数 (c) 练习2.24中的双输出函数 (d) 练习2.35中的函数 (e) 一个四输入优先编码器（见练习2.36）

练习5.64 对以下函数重复练习5.63:

- (a) 一个八输入优先编码器（见习题2.36） (b) 一个3:8译码器 (c) 一个4位进位传播加法器（无进位输入或输出） (d) 来自习题3.22的有限状态机

- (e) 来自练习3.27的格雷码计数器

练习5.65 考虑图5.60所示的Cyclone IV LE。根据数据手册，其时序规格如表5.7所示。

- (a) 实现图3.26所示FSM所需的最少Cyclone IV逻辑单元(LE)数量是多少？ (b) 无时钟偏斜时，该FSM能可靠运行的最快时钟频率是多少？ (c) 存在3 ns时钟偏斜时，该FSM能可靠运行的最快频率是多少？

表5.7 Cyclone IV时序

Name	Value (ps)
t_{pcq}, t_{ccq}	199
t_{setup}	76
t_{hold}	0
t_{pd} (per LE)	381
t_{wire} (between LEs)	246
t_{skew}	0

练习5.66 对图3.31(b)中的FSM重复练习5.65。

练习5.67 你希望使用FPGA实现一个M&M分拣器，通过颜色传感器和电机将红色糖果放入一个罐子，绿色糖果放入另一个罐子。该设计将作为有限状态机（FSM）在Cyclone IV FPGA上实现。根据数据手册，该FPGA的时序特性如表5.7所示。你希望FSM以100 MHz的频率运行。关键路径上的逻辑单元（LE）最大数量是多少？FSM能够运行的最快速度是多少？

面试问题

以下练习题展示了数字设计岗位面试中曾出现的问题。

问题5.1 两个无符号 N 位数相乘的最大可能结果是多少？

问题5.2 *Binary coded decimal (BCD)* 表示法使用四位二进制编码每个十进制数字。例如， 42_{10} 表示为 01000010_{BCD} 。请用文字解释处理器为何可能使用BCD表示法。

问题5.3 设计用于相加两个8位无符号BCD数的硬件（见问题5.2）。绘制设计原理图，并编写BCD加法器的HDL模块。输入为 A 、 B 和 C_{in} ，输出为 S 和 C_{out} 。 C_{in} 和 C_{out} 是1位进位， A 、 B 和 S 是8位BCD数。

